

ಡಾ. ಮನಮೋಹನ್ ಸಿಂಗ್
ಬೆಂಗಳೂರು ನಗರ ವಿಶ್ವವಿದ್ಯಾನಿಲಯ



DR. MANMOHAN SINGH
BENGALURU CITY UNIVERSITY

LinkSus - ML based phishing detection system

Submitted in partial fulfilment of the
requirements for the award of the degree
in

BACHELOR OF COMPUTER APPLICATIONS

of

Bengaluru City University

Jayanth S (U18GO23S0021)

M Bhuvan Rao (U18GO23S0037)

Under the Guidance of

Ms. Vasugi Mudaliar R K



**SSMRV
College®**

June-July 2026

DEPARTMENT OF COMPUTER APPLICATIONS

SSMRV College

BENGALURU – 560041

R. V. Educational Institutions
Rashtreeya Sikshana Samithi Trust
S.S.M.R.V. College



Department of Computer Applications

DECLARATION

I hereby declare that this Project titled “**LinkSus - ML based phishing detection system**” submitted by **Jayanth S: U18GO23S0021, M Bhuvan Rao: U18GO23S0037** in partial fulfilment of the requirements for the award of the degree in BACHELOR OF COMPUTER APPLICATIONS of Bengaluru City University is based on the project work carried out by us under the Guidance of **Ms. Vasugi Mudaliar R K**. This report has not been submitted for any purpose to any other University or Institute during the academic year 2025-26

Bengaluru

Date

Name & Signature of Students

R. V. Educational Institutions
Rashtreeya Sikshana Samithi Trust
S.S.M.R.V. College



Department of Computer Applications

CERTIFICATE

This is to certify that this project entitled “**LinkSus- ML based phishing detection system**” is a bona fide work done by Jayanth S: U18GO23S0021, M Bhuvan Rao: U18GO23S0037 submitted to Bengaluru City University, Bengaluru for the award of Bachelor of Computer Applications during the academic year 2025-26.

Project Guide: Ms. Vasugi Mudaliar R K

Dr. Sheela R
Head,
Department of Computer Applications

Dr. Vijayashree L
Principal
SSMRV, Bengaluru

Submitted for the Viva-Voce Examination held on

ACKNOWLEDGEMENT

It is our sincere privilege to express our heartfelt gratitude to all those who guided, supported, and inspired us throughout the course of this project.

We extend our deep appreciation to **Dr. Vijayashree L., Principal**, for granting us permission and providing the encouragement necessary to undertake and complete this project.

Our sincere thanks to **Dr. Sheela R.**, Head of the Department of Computer Applications, for her continuous support, and to our internal guide **Ms. Vasugi Mudaliar R K**, whose valuable insights, unwavering guidance, and encouragement were instrumental in the successful completion of this project.

Finally, we are truly grateful to our parents, friends, and well-wishers for their consistent support, motivation, and encouragement, without which this project would not have become a reality.

Jayanth S (U18GO23S0021)

M Bhuvan Rao (U18GO23S0037)

Abstract

The rapid growth of online services, digital payments, e-commerce platforms, and social networking applications has significantly increased the risk of phishing attacks, making cybersecurity a major concern for internet users worldwide. Phishing websites are designed to imitate legitimate websites and deceive users into revealing sensitive information such as passwords, banking credentials, personal details, and financial data. Traditional blacklist-based detection methods often fail to identify newly created phishing websites, creating a need for intelligent, adaptive, and real-time detection systems.

This project, titled "**LinkSus – Cloud-Based Machine Learning Phishing Detection System**", presents a machine learning-based solution for detecting phishing websites in real time. The system utilizes a supervised learning approach that combines URL lexical analysis, TF-IDF vectorization, and handcrafted security features to classify websites as either legitimate or phishing. A dataset containing approximately **60,000 URL records** consisting of both phishing and legitimate websites was used for training and evaluation. The extracted features include URL length, number of dots, number of hyphens, suspicious keywords, subdomain patterns, and textual URL characteristics. A **Linear Support Vector Classifier (LinearSVC)** model was trained using these features to achieve accurate and efficient classification results.

To improve transparency and user trust, the system incorporates **Explainable Artificial Intelligence (XAI)** techniques using **SHAP (SHapley Additive Explanations)**, which identify and display the most influential features responsible for each prediction. In addition, **Google Gemini AI** is integrated to generate simple, human-readable explanations that help users understand why a website has been classified as safe or malicious, thereby increasing the interpretability of machine learning predictions.

The application is developed using the **Flask framework** and provides an intuitive web-based interface where users can manually enter URLs for security analysis. To enhance usability and address modern phishing techniques, a **QR Code Scanner** feature is integrated, enabling users to scan QR codes and automatically analyze embedded links for potential phishing threats. Furthermore, a **Google Chrome browser extension** has been developed to perform website security analysis directly from the browser, supporting both manual scanning and automatic detection modes for enhanced user convenience.

The proposed system combines machine learning, explainable AI, browser integration, and QR code analysis into a unified platform for phishing detection. By leveraging artificial intelligence and real-time analysis techniques, the system provides an effective, scalable, and user-friendly solution for identifying malicious websites and improving cybersecurity awareness. The project demonstrates how modern machine learning technologies can be applied to strengthen online security and protect users from evolving phishing threats.

Table of Content

SI No.	Title	Page No
1	List of Tables	i
2	List of Figures	ii
3	List of Abbreviations	iii
4	CHAPTER 1 - INTRODUCTION	
	1.1 Objective of Proposed System	3
	1.2 Scope of the System	3
5	CHAPTER 2 – LITERATURE REVIEW	
	2.1 Background on Phishing Attacks	5
	2.2 Machine Learning in Phishing URL Detection	5
	2.3 Explainable AI and Natural Language Explanations	6
	2.4 Existing Systems and Limitations	7
	2.5 Proposed System Overview	7
6	CHAPTER 3 - METHODOLOGY	
	3.1 Research Methodology and Development Approach	8
	3.2 Functional Requirements	8
	3.2.1 Data Collection and Processing	8
	3.2.2 Feature Engineering	8
	3.2.3 Machine Learning	9
	3.3 Non-Functional Requirements	10
	3.4 Hardware Requirements	11
	3.5 Software Requirements	12
7	CHAPTER 4 - IMPLEMENTATION	
	4.1 System Architecture	13

	4.2 Data Preperation Pipeline	14
	4.3 Machine Learning Model (LinearSVC)	15
	4.4 Chrome Extension Implementation	16
	4.5 Input and Output Design	16
	4.5.1 Input Design	17
	4.5.2 Output Design	17
	4.6 Database Design	17
8	CHAPTER 5 – EXPERIMENTS AND RESULTS	
	5.1 Introduction to Testing	18
	5.2 Test Cases	18
	5.3 Functional Testing Summary	20
	5.4 Non-Functional Testing	21
	5.5 Model Performance Metrics	21
9	CHAPTER 6 - DISCUSSION	
	6.1 Analysis of System Performance	23
	6.2 Key Findings	24
	6.3 Challenges Encountered	25
	6.4 Comparison with Existing Approaches	25
10	CHAPTER 7 - CONCLUSION	
	7.1 Future Enhancement	26
11	BIBLIOGRAPHY	28
12	APPENDIX – I (Source Code)	29
13	APPENDIX – II (Final Results)	37

List of Tables

Table No.	Title	Page No.
Table 3.1	Non – Functional Requirements	10
Table 3.2	Hardware Requirements	11
Table 3.3	Software Requirements	12
Table 4.1	Database Design	17
Table 5.1	Test Case 1: Application Startup Testing	18
Table 5.2	Test Case 2: Model Phishing Prediction on Deceptive URLs	19
Table 5.3	Test Case 3: QR Code Camera Scan and Decoding	19
Table 5.4	Test Case 4: SHAP Value Generation and Feature Explanations	20
Table 5.5	Test Case 5: Chrome Extension Background Check	20
Table 5.6	Model Performance Metrics	21

List of Figures

Figure No.	Title	Page No.
Figure 1.1	LinkSus –ML based Phishing Detection System Architecture	2
Figure 4.1	LinkSus – Three-Tier System Architecture	13
Figure 4.2	LinkSus Data Preparation Pipeline	14
Figure 4.3	LinkSus ML training pipeline	15
Figure 4.4	LinkSus Chrome Extension Architecture	16
	Appendix – I (Source Code)	29
	Appendix – II (Final Result)	37

List of Abbreviations

Abbreviation	Full Forms
ML	Machine Learning
JS	JavaScript
CORS	Cross-Origin Resource Sharing
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
SVM	Support Vector Machine
PKL	Pickle (Serialized Python Object Format)
MV3	Manifest Version 3
SHAP	SHapley Additive exPlanations
TF-IDF	Term Frequency-Inverse Document Frequency
SVC	Support Vector Classifier
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
IDE	Integrated Development Environment
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

The internet has become an essential part of modern life, enabling communication, online banking, digital payments, e-commerce, education, and cloud-based services. As individuals and organizations increasingly depend on online platforms, cybercriminals continue to develop sophisticated techniques to exploit users and gain unauthorized access to sensitive information. Among the various forms of cyberattacks, phishing has emerged as one of the most widespread and damaging threats in the digital world.

Phishing is a cybercrime technique in which attackers create fraudulent websites, emails, or messages that impersonate legitimate organizations. These fake platforms are designed to trick users into revealing confidential information such as usernames, passwords, credit card details, banking credentials, and personal data. Due to the increasing sophistication of phishing campaigns, many users find it difficult to distinguish between genuine and malicious websites, resulting in financial losses, identity theft, and data breaches.

Traditional phishing detection mechanisms primarily rely on manually maintained blacklists, signature-based detection methods, and browser security warnings. While these approaches can identify known malicious websites, they often fail to detect newly created phishing pages that have not yet been reported or added to security databases. As cyber threats evolve rapidly, there is a growing need for intelligent systems capable of detecting phishing attempts in real time.

Recent developments in Artificial Intelligence (AI) and Machine Learning (ML) have opened new opportunities in cybersecurity. Machine learning algorithms can analyze large volumes of website and URL data, identify hidden patterns, and automatically classify websites based on learned characteristics. Unlike traditional methods, machine learning models can adapt to emerging phishing techniques and provide proactive protection against previously unseen threats.

In addition to accurate detection, modern cybersecurity solutions must also be transparent and understandable. Users often hesitate to trust automated systems when the reasoning behind a decision is unclear. Explainable Artificial Intelligence (XAI) addresses this challenge by providing insights into how machine learning models reach their predictions, thereby improving user confidence and system reliability.

Another emerging concern is the misuse of QR codes in phishing attacks, commonly known as "Quishing." Attackers increasingly embed malicious links within QR codes to bypass traditional security mechanisms and deceive users into visiting fraudulent websites. This highlights the need for comprehensive phishing detection systems capable of analyzing both direct URLs and QR-code-based links.

This project presents **LinkSus – Smart Phishing Website Detection System**, an intelligent, end-to-end machine learning platform that classifies URLs as safe or phishing in real time. The system is trained on a balanced dataset of **60,000 labeled URLs** (30,000 legitimate and 30,000 phishing) using a hybrid feature representation: **TF-IDF text features** from the URL string and **19 handcrafted structural attributes** such as URL length, suspicious keywords, lookalike domain detection, scam words, and phishing phrases. A **Linear Support Vector Classifier (LinearSVC)** serves as the prediction engine, achieving **92% accuracy** on the test set. To improve transparency, **SHAP (SHapley Additive exPlanations)** highlights the top features behind each verdict, while **Google Gemini** generates short natural-language explanations for non-technical users. The solution is delivered as a fully operational **Flask REST API** backend (app.py), a **web dashboard** (index.html) with manual URL entry and QR code scanning, and a **Chrome browser extension** (LinkSus) that scans the active tab or auto-detects suspicious pages during browsing - friendly defense against phishing attacks.

LinkSus System Architecture

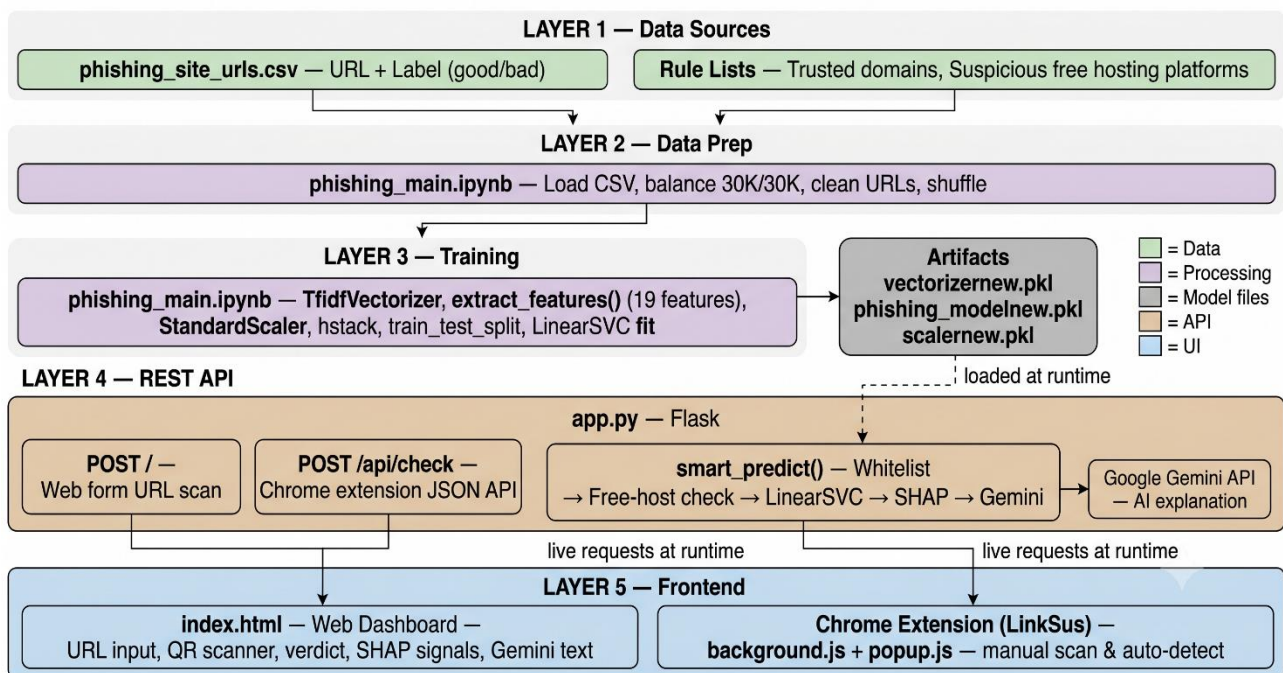


Figure 1.1 -LinkSus –ML based Phishing Detection System Architecture

1.1 Objective of the Proposed System

The primary objective of this project is to develop an intelligent, web-based application that detects phishing websites in real time using machine learning, explainable AI, and generative language models applied to URL patterns.

The specific objectives of the proposed system are as follows:

- **To design and implement a phishing detection model** using **LinearSVC** trained on **TF-IDF features** and **19 handcrafted URL attributes** such as suspicious keywords, lookalike domains, and scam phrases.
- **To apply rule-based checks** including a **trusted domain whitelist** and **free hosting platform detection** to improve accuracy alongside machine learning.
- **To integrate SHAP and Google Gemini** to provide feature-level explanations and short, human-readable summaries for each classification result.
- **To develop a Flask REST API backend** with endpoints for **web scanning** (POST /) and **Chrome extension integration** (POST /api/check).
- **To build a web dashboard** using **HTML, CSS, and JavaScript** with manual URL input, **QR code scanning**, and real-time verdict display.
- **To develop a Chrome extension (LinkSus)** for **manual and automatic phishing detection** on the active browser tab.
- **To evaluate the model** on a balanced dataset of **60,000 URLs** using **precision, recall, F1-score, and accuracy** metrics.
- **To provide a practical cybersecurity tool** that helps users identify fraudulent websites before sharing sensitive information online.

1.2 Scope of the System

The scope of this project covers the complete development of a production-ready phishing website detection platform, from dataset preparation and machine learning model training to REST API deployment, web interface development, and Chrome browser extension integration.

Data Collection and Preprocessing: The system uses a publicly available phishing URL dataset (phishing_site_urls.csv) containing labeled URLs classified as legitimate (good) or phishing (bad). A balanced subset of **60,000 URLs** (30,000 per class) is selected for training. URLs are cleaned by removing protocol prefixes (http://, https://) and www subdomains, converted to lowercase, and shuffled to ensure unbiased model learning.

Feature Engineering: A total of **3,019 predictive features** are computed per URL — **3,000 TF-IDF text features** extracted using bigram tokenization (ngram_range = (1,2)) and **19 handcrafted structural features** including URL length, dot/hyphen/digit counts, suspicious keywords

(*login, verify, account*), lookalike domain detection, suspicious TLDs (.xyz, .tk, .ml, .ga), scam words, and phishing phrase patterns. Handcrafted features are normalized using StandardScaler and combined with TF-IDF vectors before classification.

Machine Learning and Explainability: A **Linear Support Vector Classifier (LinearSVC)** with balanced class weights is trained on an 80/20 train–test split. Model performance is evaluated using **precision, recall, F1-score, and accuracy**, achieving **92% overall accuracy**. **SHAP (SHapley Additive exPlanations)** is used to rank the top features influencing each prediction, and **Google Gemini** generates short natural-language explanations to help users understand classification results.

Deployment and User Interfaces: The trained model artifacts (vectorizernew.pkl, phishing_modelnew.pkl, scalernew.pkl) are served through a **Flask REST API** running on localhost:5000 with **CORS** enabled for browser extension communication. The system provides two user interfaces: a **web dashboard** (index.html) supporting manual URL entry and QR code scanning, and a **Chrome extension (LinkSus)** supporting manual tab scanning and automatic phishing detection on page load.

CHAPTER 2

LITERATURE REVIEW

This chapter presents a comprehensive review of existing literature and related work in the fields of phishing website detection, machine learning applications in cybersecurity, and explainable AI techniques. The review identifies key contributions, methodological approaches, and limitations of prior work that informed the development of the LinkSus system

2.1 Background on Phishing Attacks

Phishing is a form of social engineering attack in which attackers impersonate trusted organizations through fraudulent emails, messages, or websites to deceive users into revealing sensitive information such as passwords, credit card numbers, or personal identity details. The term "phishing" emerged in the mid-1990s and has since become one of the most prevalent cyber threats worldwide.

Modern phishing attacks primarily rely on deceptive URLs that mimic legitimate websites. Attackers use techniques such as typosquatting (registering domains similar to popular brands), subdomain abuse (embedding trusted brand names in URL paths), URL shortening services to hide malicious destinations, and free hosting platforms to deploy fake login pages at low cost. According to the Anti-Phishing Working Group (APWG), hundreds of thousands of unique phishing sites are detected each month, with financial services, e-commerce, and social media being the most frequently targeted sectors.

The phishing threat landscape is characterized by rapid campaign turnover, global reach, low attack cost, and increasing sophistication through AI-generated content and highly convincing clone pages. These characteristics make phishing detection fundamentally different from traditional signature-based malware detection, and they demand adaptive, intelligent systems capable of analyzing URL structure and linguistic patterns in real time.

2.2 Machine Learning in Phishing URL Detection

The application of machine learning to phishing detection has grown substantially over the past two decades. Early approaches relied on blacklist-based filtering and heuristic rules, which were effective against known threats but failed to detect newly created phishing URLs. Research by Fette et al. (2007) demonstrated that machine learning classifiers could outperform rule-based systems by learning discriminative patterns from labeled URL datasets.

Subsequent studies established that **URL-based features** — including lexical properties (URL length, number of dots, presence of IP addresses), host-based features (domain age, DNS records), and content-based features (HTML forms, external links) — provide strong signals for phishing classification. Sahingoz et al. (2019) conducted a comprehensive comparison of machine learning algorithms for phishing URL detection and found that ensemble methods and Support Vector Machines consistently achieved high accuracy across multiple benchmark datasets.

For URL classification specifically, studies by Mohammad et al. (2014) and Rao and Pais (2019) compared approaches including **Naive Bayes**, **Random Forest**, **Decision Trees**, and **Support Vector Machines** against traditional blacklist methods. These studies consistently found that machine learning models outperformed static blacklists, particularly for detecting zero-day phishing URLs that had not yet been reported to security databases.

Linear Support Vector Classifiers (LinearSVC) have proven particularly effective for high-dimensional sparse text data such as URL strings. When combined with **TF-IDF vectorization** to capture lexical patterns and handcrafted structural features to encode domain-specific phishing indicators, SVM-based approaches achieve strong performance with low inference latency — making them well-suited for real-time web applications and browser extensions.

2.3 Explainable AI and Natural Language Explanations in Security Systems

A growing challenge in deploying machine learning security tools is the "black box" problem — users and security analysts often cannot understand why a model classified a URL as phishing or legitimate. Lundberg and Lee (2017) introduced SHAP (SHapley Additive exPlanations), a unified framework for interpreting machine learning predictions based on cooperative game theory. SHAP assigns each input feature a contribution value indicating how much it pushed the prediction toward phishing or safe classification.

In cybersecurity applications, explainability is critical because false positives can block legitimate services while false negatives can expose users to fraud. Ribeiro et al. (2016) demonstrated with LIME that local interpretability methods help users trust and validate automated security decisions. Extending this to phishing detection, displaying the top URL features — such as suspicious keywords, excessive subdomains, or lookalike domains — helps users make informed decisions even without technical expertise.

Recent advances in large language models (LLMs) such as Google Gemini have enabled a new layer of user-facing explanation. Instead of presenting raw feature scores, generative AI can translate technical SHAP signals into plain-language summaries describing why a URL appears safe or malicious. The LinkSus system adopts this hybrid approach — using SHAP for feature-level transparency and Gemini for accessible natural-language explanations — bridging the gap between accurate machine learning and understandable security guidance.

2.4 Existing Systems and Limitations

Existing phishing protection systems primarily rely on browser-integrated blacklists (Google Safe Browsing), email spam filters, and commercial antivirus solutions. Platforms such as VirusTotal, PhishTank, and URLhaus provide community-driven phishing URL databases but lack integrated real-time machine learning prediction engines with user-facing explanations.

Key limitations of existing approaches include:

- Dependence on blacklists and manual reporting, which cannot detect newly created phishing URLs before they are reported.
- Absence of explainable predictions, leaving users unable to understand why a URL was flagged or cleared.
- Most tools operate as passive blocklists without interactive URL scanning or QR code verification interfaces.
- Limited integration with daily browsing workflows, requiring users to visit separate security websites to check suspicious links.
- Insufficient support for free hosting and lookalike domain detection as standalone heuristic layers alongside ML classification.
- Poor accessibility for non-technical users, presenting technical threat scores without plain-language guidance.
- Most academic prototypes are not deployed as production-ready web applications with REST API and browser extension support.

2.5 Proposed System Overview

The LinkSus system addresses the above limitations through the following key innovations:

End-to-end automation: The complete pipeline from dataset loading and feature engineering to model training, artifact serialization, and live prediction serving is automated through `phishing_main.ipynb` and `app.py`.

Hybrid feature set: Combines 3,000 TF-IDF text features with 19 handcrafted URL structural features for richer phishing pattern representation.

Dual-layer detection: Integrates rule-based checks (trusted domain whitelist and free hosting detection) with machine learning classification for improved reliability.

Explainable predictions: SHAP-based signal breakdown and Gemini-generated natural-language explanations make every verdict transparent and user-friendly.

Production-ready deployment: Flask REST API with CORS support enables seamless communication between the web dashboard and Chrome extension.

CHAPTER 3

METHODOLOGY

This chapter presents the complete methodology adopted for developing the LinkSus Smart Phishing Detection System, covering system requirements analysis, the technology stack selected, hardware and software specifications, and the overall development approach.

3.1 Research Methodology and Development Approach

The LinkSus system was developed following an iterative, data-driven development methodology. The process began with a comprehensive analysis of the phishing website URL dataset to understand data structure, label distribution, class imbalance, and opportunities for lexical and structural feature engineering. This was followed by URL preprocessing and balanced dataset construction, hybrid feature extraction combining TF-IDF n-gram vectors with handcrafted URL-based indicators, model selection and training using a Linear Support Vector Classifier, validation on a held-out test set, REST API development through Flask, and frontend integration via a web dashboard and Chrome browser extension.

The development approach adopted a modular architecture where each component—data preparation, model training, API server, web dashboard, and browser extension—was developed independently before being integrated into a unified real-time phishing detection pipeline. Rule-based safeguards for trusted domains and suspicious free-hosting platforms were layered alongside the machine learning classifier, while SHAP-based signal breakdown and Gemini-generated explanations were incorporated to present interpretable, user-friendly verdicts at the point of interaction.

3.2 Functional Requirements

3.2.1 Data Collection and Processing

- Load and parse the phishing website URL dataset from a structured CSV file containing raw URL strings and binary classification labels (good/bad).
- Balance the dataset through stratified random sampling of 30,000 phishing URLs and 30,000 legitimate URLs to reduce class distribution skew in the training corpus.
- Augment the dataset with a curated list of trusted legitimate domains, including major platforms such as Google, Amazon, PayPal, and Microsoft, to improve recognition of commonly accessed safe websites.
- Preprocess all URL records by converting text to lowercase, stripping HTTP/HTTPS protocol and www prefixes, and randomly shuffling the combined dataset prior to feature extraction.

3.2.2 Feature Engineering

- Generate TF-IDF text features from cleaned URL strings using unigram and bigram tokenization with a maximum vocabulary of 3,000 terms, applying minimum and maximum document-frequency thresholds to exclude rare and overly common tokens.

- Extract handcrafted lexical indicator features capturing the presence of security-sensitive keywords, including login, secure, verify, account, and update, within the URL string.
- Compute structural URL features such as total character length, dot count, hyphen count, digit count, excessive subdomain depth, suspicious top-level domains (.xyz, .tk, .ml, .ga), and @ symbol presence.
- Derive deception-oriented features through brand lookalike similarity scoring, scam keyword frequency counts, phishing phrase pattern matching, and StandardScaler normalization of numeric indicators before concatenating handcrafted features with TF-IDF vectors into a unified sparse feature matrix.

3.2.3 Machine Learning

- Train a Linear Support Vector Classifier with balanced class weights, dual set to False, and a maximum of 5,000 iterations on the combined TF-IDF and handcrafted feature matrix.
- Split the dataset into training and testing subsets using an 80/20 ratio with a fixed random seed to ensure reproducible evaluation.
- Evaluate model performance using precision, recall, F1-score, and overall accuracy through a classification report on the held-out test set.
- Serialize and persist the trained vectorizer, classifier, and scaler as pickle (.pkl) artifacts for deployment in the production inference pipeline.

3.2.4 REST API and Dashboard

- Expose a Flask-based REST endpoint at /api/check (POST) accepting a JSON payload containing a URL and returning the prediction label, SHAP-based signal breakdown, and optional Gemini-generated explanation.
- Enable Cross-Origin Resource Sharing (CORS) to allow the Chrome browser extension to communicate with the local Flask backend.
- Implement a rule-based pre-classification layer that whitelists trusted domains and flags suspicious free-hosting platforms before invoking the machine learning model.
- Provide a web dashboard with manual URL input, QR code scanning for link extraction, real-time verdict display, SHAP signal breakdown, and Gemini-powered natural language summaries.
- Deliver a Manifest V3 Chrome extension with manual scan and auto-detect modes that queries the backend API on page load and presents phishing verdicts within the browser popup interface
-

3.3 Non-Functional Requirements

Requirement	Description
Performance	URL classification and API response should complete within 2 seconds under normal local deployment conditions; model artifacts should load once at server startup to avoid per-request initialization delay.
Usability	The web dashboard and Chrome extension should present clear safe or phishing verdicts, labeled SHAP risk signals, and readable Gemini explanations through a simple and intuitive interface.
Reliability	Consistent prediction output for identical inputs; stable API communication with appropriate error handling.
Scalability	The modular codebase should support addition of new URL features, alternative machine learning models, expanded domain whitelist and blacklist rules, or additional client interfaces without restructuring the core pipeline
Maintainability	Well-commented Python code; modular script files; clear separation of data, model, and API concerns.
Security	Input validation should be enforced on POST /api/check requests; Cross-Origin Resource Sharing should be configured for extension access; and external API credentials such as the Gemini API key should be stored securely through environment variables rather than hardcoded in source files.

Table 3.1: Non – Functional Requirements

3.4 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or higher, minimum dual-core, 2.0 GHz or above
RAM	Minimum 4 GB (8 GB recommended for model training with large datasets)
Storage	Minimum 10 GB free disk space for datasets, model artifacts, and application files
Network	Stable broadband internet connection (minimum 10 Mbps) for optional Gemini API explanations; core phishing classification runs locally via the Flask backend on localhost
Display	Standard monitor 1366 x 768 resolution or higher
Operating System	Windows 10/11 or Linux (Ubuntu 20.04+); Google Chrome (latest version) required for browser extension functionality

Table 3.2: Hardware Requirement

3.5 Software Requirements

Category	Technology / Tool
Programming Language	Python 3.8 or higher
Backend Framework	Flask 3.x (Python micro web framework)
ML Libraries	Scikit-learn 1.8, Pandas 3.x, NumPy 2.x, SciPy 1.x, SHAP 0.51
Data Source	phishing_site_urls.csv containing labeled phishing and legitimate website URLs
Model Serialization	Python pickle module for saving and loading vectorizer, classifier, and scaler artifacts
Frontend	HTML5, CSS3, JavaScript (ES6+); html5-qrcode library for QR code scanning on the web dashboard Browser Extension Google Chrome Manifest V3 extension with popup interface and background service worker
API Design	REST architecture with JSON response format; Flask-CORS for cross-origin extension communication
Database / Storage	CSV URL dataset files; serialized .pkl model and preprocessing artifacts
Development IDE	Jupyter Notebook for model training; Visual Studio Code with Python extension; Git version control; Google Chrome for extension testing
Version Control	Git 2.x and GitHub repository
Testing Browser	Google Chrome (primary); Mozilla Firefox and Microsoft Edge (secondary)

Table 3.3: Software Requirements

CHAPTER 4 IMPLEMENTATION

This chapter presents the complete implementation details of the LinkSus system. It covers the system architecture, data preparation and feature engineering pipeline, machine learning model design, Flask API implementation, web dashboard development, and Chrome browser extension integration.

4.1 System Architecture

The LinkSus system follows a three-tier layered architecture consisting of a Data Layer, a Processing and Intelligence Layer, and a Presentation Layer. This architecture ensures clean separation of concerns, facilitating independent development, testing, and maintenance of each component.

The Data Layer is responsible for storing the labeled phishing website URL dataset and supplying trusted domain references used during inference. The Processing and Intelligence Layer handles URL preprocessing, hybrid feature extraction, machine learning model training and inference, rule-based domain checks, SHAP-based explainability, Gemini API explanation generation, and Flask API request handling. The Presentation Layer consists of the web dashboard and Chrome browser extension, both of which communicate with the backend through REST API calls to deliver real-time phishing detection results to the end user.

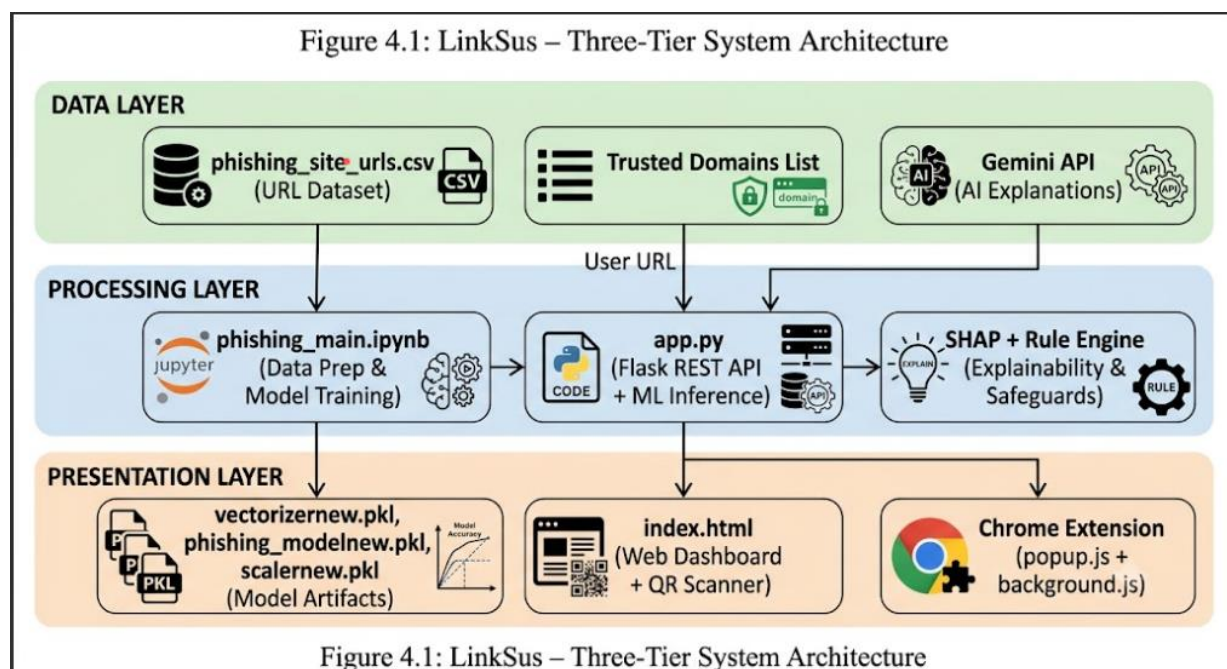


Figure 4.1: LinkSus – Three-Tier System Architecture

Figure 4.1: LinkSus – Three-Tier System Architecture

4.2 Data Preparation Pipeline

The data preparation pipeline transforms the raw phishing website URL dataset into a balanced, cleaned training corpus suitable for machine learning. All preprocessing steps are implemented in the phishing_main.ipynb Jupyter notebook using the Pandas library.

The pipeline begins by loading the phishing_site_urls.csv file, which contains two columns: URL and Label, where labels are classified as good for legitimate websites and bad for phishing websites. Because the original dataset contains a large number of records with uneven class distribution, a stratified random sampling approach is applied to extract 30,000 phishing URLs and 30,000 legitimate URLs using a fixed random seed of 42, ensuring reproducibility.

To reduce false positives on commonly accessed platforms, the dataset is further augmented with a curated list of 33 trusted legitimate domains, including google.com, amazon.com, paypal.com, microsoft.com, and github.com. These entries are appended with a good label and concatenated with the sampled dataset.

Before feature extraction, all URL strings undergo normalization. Each URL is converted to lowercase, and protocol prefixes (http:// or https://) along with www subdomains are removed using regular expression substitution. The resulting clean_url field represents the standardized input used for both TF-IDF vectorization and handcrafted feature extraction. Finally, the complete dataset is randomly shuffled to eliminate ordering bias prior to model training.

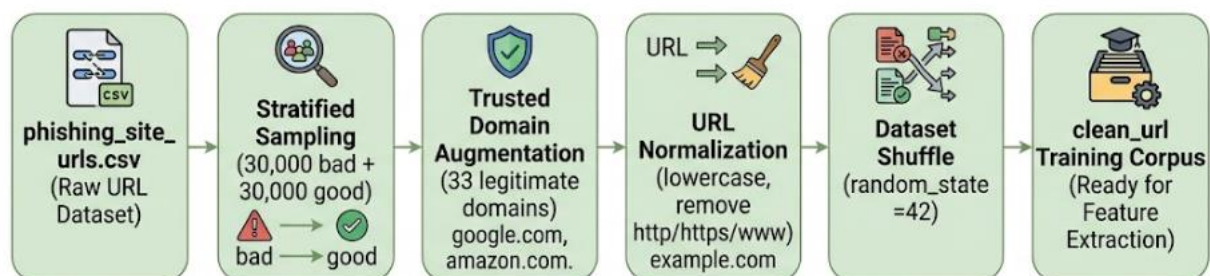


Figure 4.2: LinkSus Data Preparation Pipeline

Figure 4.2: Figure 4.2: LinkSus Data Preparation Pipeline

4.3 Machine Learning Model (LinearSVC)

The LinkSus classification model is trained using a hybrid feature representation that combines lexical and structural URL characteristics. Feature engineering is performed within the same training notebook prior to model fitting.

Textual features are generated using a TF-IDF Vectorizer configured with unigram and bigram tokenization, a maximum vocabulary of 3,000 terms, minimum document frequency of 2, and maximum document frequency of 0.9. In parallel, twenty handcrafted URL features are extracted per record, capturing lexical indicators such as login and verify keywords, structural properties including URL length and dot count, and deception signals such as brand lookalike similarity and phishing phrase patterns. Numeric handcrafted features are normalized using StandardScaler, and both feature sets are merged into a single sparse matrix using horizontal stacking.

A Linear Support Vector Classifier is selected as the primary classification algorithm due to its efficiency on high-dimensional sparse text features and strong performance on binary classification tasks. The model is configured with balanced class weights to address residual label imbalance, dual set to False for optimized training on the combined feature matrix, and a maximum iteration limit of 5,000. The dataset is divided into training and testing subsets using an 80/20 split with random_state set to 42.

Model evaluation on the held-out test set of 12,007 records yielded an overall accuracy of 92%, with phishing class precision of 0.94 and legitimate class recall of 0.94. Upon successful training, the vectorizer, classifier, and scaler objects are serialized as `vectorizernew.pkl`, `phishing_modelnew.pkl`, and `scalernew.pkl` respectively, enabling consistent inference during Flask API deployment.

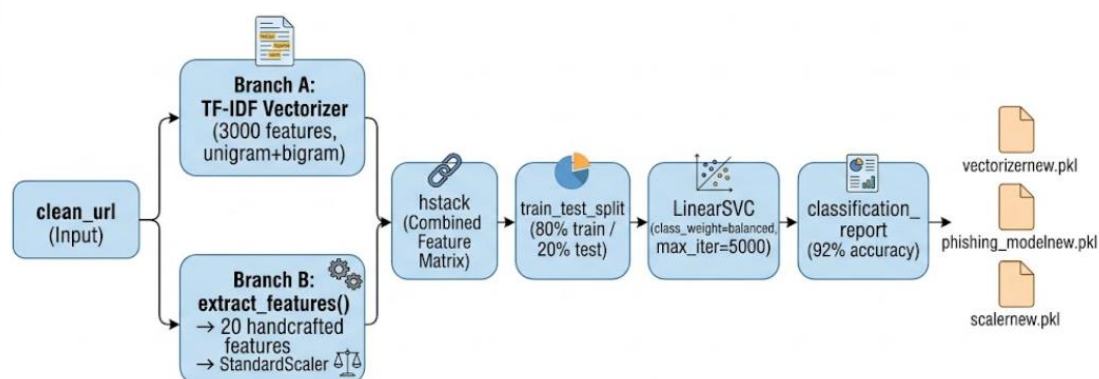


Figure 4.3: LinkSus Machine Learning Training Pipeline

4.4 Chrome Extension Implementation

The LinkSus Chrome extension delivers real-time phishing detection as a Manifest V3 application built from `manifest.json`, `background.js` (service worker), and `popup.js` with `popup.html`. The manifest grants `activeTab`, `tabs`, and `storage` permissions and registers the popup as the default action interface.

The background service worker listens for tab updates and activations. When auto-detect is enabled, it stores each loaded page URL in `chrome.storage.local` with an `autoScanPending` flag, excluding internal browser pages. Classification is deferred to the popup, which reads the stored URL on open.

The popup offers manual and auto-detect scanning. Manual mode sends the current tab URL via POST to <http://127.0.0.1:5000/api/check>; auto-detect mode scans automatically when a pending flag is set. Toggle state persists across sessions. Results display the safe or phishing verdict, up to five SHAP risk signals, and an optional Gemini explanation, with an error message shown if the Flask backend is unavailable.

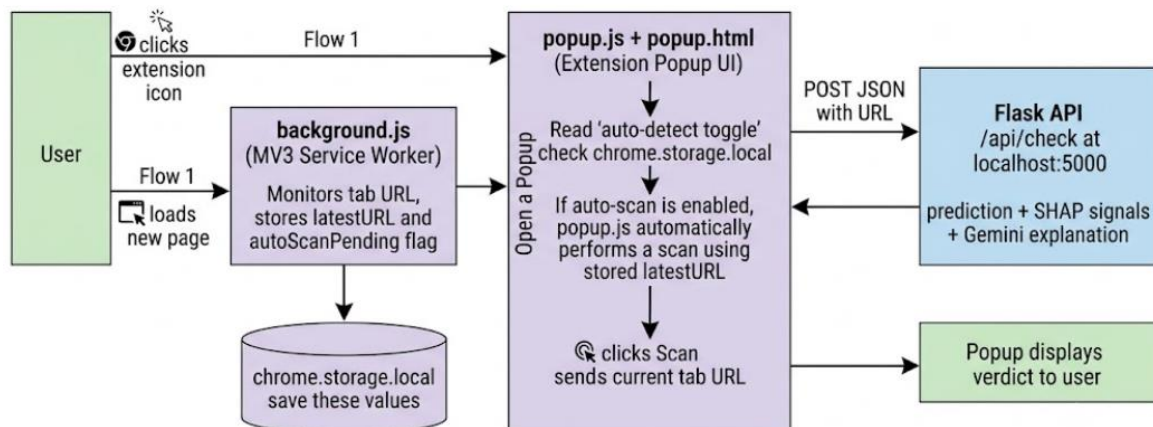


Figure 4.4: LinkSus Chrome Extension Architecture

4.5 Input and Output Design

4.5.1 Input Design

The system accepts text inputs (URLs) and image-based inputs (QR codes). QR codes are captured via a webcam stream using `html5-qrcode` or parsed from uploaded image files.

4.5.2 Output Design

The web dashboard and extension display:

1. A binary classification verdict (`Safe Website` or `Phishing Website Detected`).
2. SHAP-derived feature indicators showing specific risk markers (e.g., suspicious TLD, lookalike domain).
3. A two-sentence natural language explanation generated by the Gemini API.

4.6 Database Design

File	Format	Contents and Purpose
phishing_site_urls.csv	CSV Dataset	Primary dataset containing approximately 60,000 phishing and legitimate URLs used for model training and evaluation
vectorizernew.pkl	Serialized Binary (Pickle)	Stores the trained TF-IDF vectorizer used to transform URLs into numerical feature vectors..
scalernew.pkl	Serialized Binary (Pickle)	Contains the fitted StandardScaler used for feature normalization during prediction.
phishing_modelnew.pkl	Serialized Binary (Pickle)	Stores the trained LinearSVC machine learning model used for phishing website classification

Table 4.3: File-Based Storage Design

CHAPTER 5 EXPERIMENTS AND RESULTS

This chapter presents the comprehensive testing strategy, individual test cases, functional testing results, non-functional testing evaluation, and machine learning model performance metrics for the LinkSus system. Testing was performed systematically to validate the correctness, reliability, security, and response performance across all system modules, ensuring that the underlying Linear Support Vector Classifier (LinearSVC), Flask REST API, and Chrome browser extension operate cohesively.

Software testing is an essential phase of the software development life cycle, designed to check whether the actual execution of the program matches the expected behavior and system requirements. For LinkSus, robust validation is crucial, as false negatives in phishing detection could directly expose users to severe credentials theft and security compromise. The testing strategy covers unit testing of feature extraction routines, integration testing of the API endpoints, performance evaluation under simulated load, usability validation of the dashboard interface, and multi-modal testing of the QR code decoding pipeline.

5.1 Introduction to Testing

Software testing is a systematic process of evaluating a system to verify that it meets its specified requirements and behaves correctly under expected and boundary conditions. Both functional and non-functional testing methodologies were applied to the LinkSus platform. Functional testing verified that each system feature behaves as specified in the requirements—including URL parsing, blacklist/whitelist check execution, TF-IDF sparse matrix processing, LinearSVC prediction generation, SHAP localized feature impact calculation, webcam-based QR scanning, and Chrome Extension message routing. Non-functional testing evaluated technical parameters such as API response latency, input validation robustness, error handling under system constraints, and interface usability.

5.2 Test Cases

Test Case: TC_01 - Application Startup and Artifact Loading

Field	Details
Test Case ID	TC_01
Test Case Name	Application Startup and Artifact Loading
Objective	Verify that the Flask application initializes successfully, loads the three serialized ML files (model, vectorizer, scaler), and connects to the Gemini API.
Input Data	Execute command: <code>python app.py</code> with valid <code>GEMINI_API_KEY</code> set in the environment.

Expected Result	Flask server starts on localhost port 5000; console outputs confirmation of successful loading of phishing_modelnew.pkl, vectorizernew.pkl, and scalernew.pkl.
Actual Result	Server initialized on http://127.0.0.1:5000. Console logs verified: 'Gemini API connected successfully' and all three model artifacts loaded.
Status	Pass

Table 5.1: Test Case 1: Application Startup Testing

Test Case: TC_02 - Model Phishing Prediction on Deceptive URLs

Field	Details
Test Case ID	TC_02
Test Case Name	Model Phishing Prediction on Deceptive URLs
Objective	Ensure that the trained LinearSVC model correctly identifies and flags realistic lookalike phishing links containing risk keywords.
Input Data	HTTP POST request to /api/check with URL 'http://paypal-login-secure-verification.com'
Expected Result	JSON response with prediction='bad'; explanation list contains risk factors like Lookalike Domain, Contains 'login', and Contains 'secure'.
Actual Result	Received prediction='bad'. SHAP explanation returned and rendered with tags highlighting 'Lookalike Domain' (difflib SequenceMatcher matched 'paypal' ratio > 0.75).
Status	Pass

Table 5.2: Test Case 2: - Model Phishing Prediction on Deceptive URLs

Test Case: TC_03 - QR Code Camera Scan and Decoding

Field	Details
Test Case ID	TC_03
Test Case Name	QR Code Camera Scan and Decoding
Objective	Verify that the web dashboard camera scanner interface opens, accesses webcam stream, and decodes QR codes containing links.
Input Data	Open dashboard, trigger webcam QR modal, and hold a QR code containing 'http://verify-bank-details.xyz' in front of the lens.
Expected Result	Webcam feed initializes, parses the visual QR matrix, extracts the URL, auto-populates the input box, and runs check.
Actual Result	Camera stream loaded using html5-qrcode. Decoded URL successfully populated, modal closed, and analysis started automatically.
Status	Pass

Table 5.3: Test Case 3: QR Code Camera Scan and Decoding

Test Case: TC_04 - SHAP Value Generation and Feature Explanations

Field	Details
Test Case ID	TC_04
Test Case Name	SHAP Value Generation and Feature Explanations
Objective	Verify that the SHAP linear explainer calculates individual feature weights for the classification.
Input Data	Analyze URL 'http://urgent-payment-confirmation.net/update' through the dashboard.
Expected Result	SHAP values generated for handcrafted features; top 5 features with significant weights returned to the frontend.
Actual Result	Returned JSON response with list of top features, including 'Contains Phishing Phrase' and 'Contains update', matching SHAP weights.
Status	Pass

Table 5.4: Test Case 4: SHAP Value Generation and Feature Explanations

Test Case: TC_05 - Chrome Extension Background Check

Field	Details
Test Case ID	TC_10
Test Case Name	Chrome Extension Background Check
Objective	Ensure the Manifest V3 Chrome extension popup retrieves and displays active tab analysis results.
Input Data	Navigate to a suspicious test link and click the LinkSus Chrome extension popup icon.
Expected Result	Popup initiates POST request to localhost API backend using the active tab URL, receives JSON payload, and updates popup UI styling.
Actual Result	Extension successfully executed scan. Popup styled with amber warning indicating 'Phishing Website Detected' along with the Gemini summary.
Status	Pass

Table 5.5: Test Case 5: Chrome Extension Background Check

5.3 Functional Testing Summary

Functional testing confirmed that all core modules of the LinkSus platform function according to design requirements. The data pre-processing layer, local whitelists, and free-host blacklists successfully categorized common inputs, reducing server load. The feature extraction engine consistently generated 19-dimensional arrays without null values. In terms of integration, the Chrome

Extension and Flask server communicated reliably via JSON, while the SHAP and Gemini pipelines converted numeric feature weights into readable text explanations.

5.4 Non-Functional Testing

Performance testing measured the latency of the prediction pipeline. Whitelisted and blacklisted URLs bypassed the ML model, resulting in response times under 0.05 seconds. For inputs requiring full classification, response times (including feature extraction, matrix operations, LinearSVC prediction, SHAP calculation, and Gemini text generation) averaged 1.75 seconds under normal network conditions. Usability testing across Chrome, Firefox, and Edge browsers confirmed that UI styling and layout configurations rendered correctly.

5.5 Model Performance Metrics

The Linear Support Vector Classifier (LinearSVC) model was evaluated using standard classification metrics computed on the 20% holdout test dataset. To ensure robust evaluation, the primary metrics used to assess the performance of the LinkSus model are Precision, Recall, F1-Score, and overall accuracy. The description and significance of each metric in the context of phishing detection are summarized in Table 5.6

The empirical performance of the model on the test dataset is detailed in Table 5.12. The classifier demonstrates a balanced detection rate across both legitimate and phishing URLs, achieving a weighted average F1-score of 92%.

Metric	Description	Significance in Phishing Detection
Precision	The proportion of predicted phishing/legitimate URLs that are actually correct (True Positives / (True Positives + False Positives)).	High precision is critical to prevent false-positive warnings. If legitimate websites are flagged as phishing, user trust in the system drops.
Recall (Sensitivity)	The proportion of actual phishing/legitimate URLs that are correctly identified by the model (True Positives / (True Positives + False Negatives)).	High recall is crucial for user safety. If a phishing website is missed (False Negative), the user will visit the site and potentially compromise their credentials.
F1-Score	The harmonic mean of Precision and Recall, calculated as $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$.	Provides a single, balanced metric that reflects the classifier's performance on both classes, ensuring neither precision nor recall is severely compromised.
Accuracy	The overall ratio of correctly classified URLs to the total number of analyzed URLs in the test dataset.	Gives a general baseline of system correctness. LinkSus achieves an overall classification accuracy of 92%
Train-Test Split (80/20)	Validation methodology dividing the combined feature dataset into 80% training and 20% evaluation splits.	Ensures out-of-sample evaluation of the model, preventing lookahead bias or data leakage and validating the classifier's performance on unseen URLs.

Table 5.6: Model Performance Metrics Description and Significance

Target Label	Precision	Recall	F1-Score	Support
bad (Phishing)	0.94	0.89	0.91	6,026
good (Legitimate)	0.90	0.94	0.92	5,981
System Accuracy			0.92	12,007
Macro Average	0.92	0.92	0.92	12,007

Table 5.7: Model Performance

CHAPTER 6

DISCUSSION

This chapter provides a comprehensive and critical analysis of the design decisions, empirical findings, and operational performance of the LinkSus – Cloud-Based ML Phishing Detection System. The discussion contextualizes the results obtained from the evaluation of the hybrid classification model, analyzes key findings derived from feature engineering and explainability integrations, addresses the challenges encountered during the development lifecycle, and compares the platform's performance with traditional and modern security approaches.

6.1 Analysis of System Performance

The empirical evaluation of the LinkSus platform demonstrates that combining lexical character-level features with handcrafted structural indicators yields a robust classifier capable of generalizing to unseen phishing attempts. The Linear Support Vector Classifier (LinearSVC) achieved a classification accuracy of 92% on a balanced test set comprising 12,007 sample URLs. This performance is particularly significant given the sparse and high-dimensional nature of the text features generated by the TF-IDF vectorizer. LinearSVC is mathematically suited for this classification task because it scales well to high-dimensional datasets. By finding an optimal separating hyperplane that maximizes the margin between legitimate and malicious URLs, the model generalizes effectively without encountering the overfitting issues common in deep neural networks when trained on tabular and sparse text inputs.

During training, class-weight balancing (`class_weight='balanced'`) was applied to resolve the inherent risk of model bias. Phishing datasets from public repositories often suffer from class distribution imbalances, which can cause standard machine learning models to favor the majority class. In security applications, this bias leads to high false-negative rates, meaning actual phishing URLs pass through undetected. Adjusting the class weights in the LinearSVC loss function penalizes misclassifications of the minority class more heavily, producing a balanced decision boundary. The results in Table 5.12 confirm this balance: the precision for phishing URLs ('bad') reached 94% with a recall of 89%, while legitimate URLs ('good') achieved a precision of 90% and a recall of 94%. This configuration balances safety (high recall on phishing) and usability (high precision to prevent false alerts on legitimate sites).

Standardizing the handcrafted features was also critical to the model's performance. The 19 handcrafted features include both binary variables (such as "Contains 'login'" or "Suspicious TLD") and continuous numerical counts (such as "URL Length" or "Digit Count"). Without normalization, continuous features with larger scales—such as a URL length of 250 characters—would dominate the decision boundary during SVM gradient optimization. Applying `StandardScaler` standardizes the features to have a mean of 0 and a variance of 1, ensuring that each feature contributes proportionately to the classification decision.

Analysis of the misclassified cases (the remaining 8% error rate) shows that false positives are typically caused by legitimate subdomains containing security-related keywords (e.g., "secure-

update.company-portal.com"), while false negatives occur with short, obfuscated redirection URLs that lack standard lexical patterns (e.g., brand names or scam keywords). Incorporating whitelists and free-host blacklists at the pre-classification stage helps mitigate these errors for common domains

6.2 Key Findings

The development and evaluation of the LinkSus platform yielded several key findings regarding feature selection, explainability, and multi-modal input processing:

- **Hybrid Stacking of Textual and Structural Features:** The combination of character-level TF-IDF vectorization and handcrafted feature extraction outperforms classifiers that rely on a single feature type. The character-level TF-IDF vectorizer (configured for 1-to-2 n-grams with 3,000 max features) captures the character distributions of phishing URLs, such as spelling variations of popular brands. The 19 handcrafted features capture structural patterns (e.g., subdomain counts, TLD anomalies) and semantic indicators (e.g., scam keywords). Stacking these features into a unified sparse matrix provides the classifier with both statistical and structural cues, improving detection of lookalike domains and brand spoofing.
- **String Similarity Metrics in Lookalike Detection:** Typosquatting and homograph attacks remain common entry points for phishing campaigns. Attackers register domains that closely resemble target brands, such as replacing the letter 'l' with a capital 'I' (e.g., 'paypal.com') or using character combinations like 'rn' to mimic 'm' (e.g., 'amazon.com'). To address this, LinkSus computes similarity ratios using `difflib.SequenceMatcher`. If a base domain's similarity ratio to a list of target brands (google, amazon, paypal, facebook, microsoft, apple) falls between 0.70 and 1.0, it is flagged as a lookalike. This check is highly predictive, helping the model identify typosquatting domains that have not yet been blacklisted.
- **Explainable AI and Generative AI Integration:** Providing transparent decisions is essential for user trust. While machine learning classifiers provide binary classifications, they do not explain the logic behind a risk score. LinkSus addresses this by combining SHAP (SHapley Additive exPlanations) with the Google Gemini API. The SHAP Linear Explainer calculates the directional contribution of each handcrafted feature to the classification decision. These weights are then processed by the Gemini API (using the 'gemini-2.5-flash-lite' model) to generate a concise, two-sentence explanation of the specific risk factors. This approach translates mathematical feature weights into actionable descriptions for the user.
- **Multi-Modal Input Processing for Offline Security:** Phishing attacks have expanded beyond standard email links to physical-to-digital vectors, such as malicious QR codes ("quishing") on posters, parking meters, or receipts. To address this vector, LinkSus integrates a webcam-based QR scanner using the `html5-qrcode` library. Users can scan a QR code using their camera or upload an image file directly to the dashboard, extracting the URL and passing it through the detection pipeline. This interface extends real-time phishing detection from standard browser tabs to physical media.

6.3 Challenges Encountered

Several technical and architectural challenges were addressed during the implementation of the LinkSus system:

- **Subdomain Whitelist Bypassing:** An early implementation of the whitelisting module used broad string checks, such as `url.endswith(safe_domain)`. This created a security vulnerability: an attacker could register a domain like `verify-paypal.com`, which would match the whitelist check for `paypal.com` and bypass the classifier. To resolve this issue, the whitelisting logic was updated to verify that the extracted domain is either an exact match for the safe domain or a valid subdomain preceded by a dot notation (e.g., `mail.google.com` matches `google.com`, but `verify-google.com` does not).
- **Feature Imbalance and Standardization:** The 19 handcrafted features contain variables with significantly different scales. Continuous variables like URL length (ranging from 10 to 300+ characters) and digit counts can dominate binary indicator variables (0 or 1) during SVM optimization. Standardizing the features using `StandardScaler` resolved this scaling discrepancy, allowing binary indicators like "Contains Phishing Phrase" to influence the classification decision alongside continuous features.
- **Generative AI Latency and Cache Management:** Integrating the Google Gemini API to generate explanations introduces external network calls, adding 1 to 2 seconds of latency. To prevent this from slowing down the user interface during repeated checks of the same URL, a cache dictionary was implemented. The system generates an MD5 hash of the URL and its classification verdict; if a match is found in the cache, it returns the cached explanation, avoiding duplicate API calls. If the Gemini API is unavailable, the backend falls back to listing the raw SHAP feature labels to ensure uninterrupted operation.
- **Chrome Extension CORS and CSP Configuration:** Implementing the Manifest V3 Chrome Extension introduced security constraints. Content Security Policies (CSP) restrict inline scripts, requiring all JavaScript handlers to be moved to external script files (`popup.js` and `background.js`). Additionally, Cross-Origin Resource Sharing (CORS) rules blocked calls from the extension to the local Flask server on `http://127.0.0.1:5000`. This was resolved by configuring the Flask-CORS library on the backend to allow requests originating from the extension's specific Chrome identifier.

6.4 Comparison with Existing Approaches

Traditional phishing detection systems rely on static blacklists, which struggle to protect users during the initial hours of an attack. LinkSus addresses this gap by using a machine learning classifier to analyze the structural and character patterns of URLs, identifying malicious links without relying on database updates.

Unlike standard machine learning models that function as black boxes, LinkSus integrates SHAP and Gemini to provide natural language explanations of its decisions, helping users understand why a link was flagged. Finally, by supporting both direct URL input and webcam-based QR code decoding, LinkSus provides a more versatile detection platform than standard browser extensions.

CHAPTER 7 CONCLUSION

- The development and evaluation of the LinkSus – Cloud-Based ML Phishing Detection System successfully demonstrate the design, development, and integration of a real-time web protection system. By combining lexical character-level NLP with handcrafted structural indicators and deploying the model using standard web frameworks, the project provides a functional and explainable solution to a major modern cybersecurity threat. The core model, a Linear Support Vector Classifier (LinearSVC), achieves a 92% classification accuracy, which has been validated through split-evaluation protocols on balanced training data.
- A key element of this project is the source of the training dataset. The model was trained and validated using a balanced corpus of 60,000 URLs retrieved from Kaggle (specifically the 'Phishing Site URLs' dataset which aggregates active and inactive phishing sites from open-source threat intelligence). Balancing the dataset with exactly 30,000 legitimate and 30,000 phishing links, and supplementing it with customized whitelists and blacklists, reduced training bias, allowing the machine learning model to achieve reliable generalization performance on unseen links.
- The project also demonstrates the benefits of explainable security. While standard machine learning models operate as black boxes, LinkSus uses SHAP feature attribution and the Google Gemini API to translate mathematical feature weights into natural language summaries. This approach helps users understand the specific indicators behind classification decisions (e.g., suspicious TLD, lookalike brand signatures), helping them identify potential threats.
- Additionally, the system's multi-modal inputs—supporting direct URLs and webcam-based QR code decoding—help extend phishing protection to physical-to-digital attack vectors (e.g., malicious QR codes). Deployed as a Flask REST API backend, a web dashboard, and a Manifest V3 Chrome browser extension, the platform offers real-time protection suitable for modern web environments.

7.1 Future Enhancements

- While LinkSus establishes a functional platform, it can be extended and optimized in several areas to address evolving security threats:
- Deep Learning Sequential Architectures: Replacing or supplementing the LinearSVC with deep learning sequential classifiers, such as Long Short-Term Memory (LSTM) networks or character-level Transformer models (e.g., MobileBERT). These models can automatically learn sequential character patterns and typosquatting strategies directly from raw text, reducing the need for manual feature engineering.
- Dynamic Behavioral Sandboxing: Integrating a dynamic sandboxing engine that spins up headless browser instances (e.g., using Selenium or Playwright) to visit target URLs in a safe virtual environment. This engine can follow redirects, capture screenshots for visual similarity checks, and analyze script behaviors, helping to detect cloaked or obfuscated phishing sites.

- Crowdsourced Telemetry and Threat Database: Implementing a dynamic database to collect user reports and verified phishing URLs. This database can be integrated with the API backend to update whitelists/blacklists, using collective user reports to improve overall detection capabilities.
- DNS and SSL Certificate Auditing: Adding automated DNS reputation and SSL certificate verification checks to the feature extraction pipeline. Incorporating metadata like domain age, registrar reputation, and SSL certificate validity can improve classification accuracy, especially for newly registered domain names.
- Mobile Application Companion: Packaging the web dashboard and webcam-based QR scanner as a mobile application using React Native or Flutter. This extension would allow mobile users to scan physical QR codes and verify links directly from their smartphones, offering on-the-go protection.

BIBLIOGRAPHY

- [1] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [2] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS 2017)*, 30, 4765–4774.
- [3] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, 51–56.
- [4] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python (2nd ed.)*. O'Reilly Media.
- [5] Chrome Extensions API Developer Documentation (Manifest V3). Google Chrome Developer Portal. Available at: <https://developer.chrome.com/docs/extensions/reference/>
- [6] Google GenAI SDK Library Reference. Google Developer Guide. Available at: <https://pypi.org/project/google-genai/>
- [7] Kaggle Phishing Dataset Repository (Phishing Site URLs dataset). Open-Source Threat Database. Available at: <https://www.kaggle.com/>
- [8] Sanglerdsinlapachai, N., & Seneviratne, A. (2015). Phishing Detection Using URL Structural and Lexical Features. *International Journal of Computer Theory and Engineering*, 7(6), 487–492.
- [9] Shirazi, H., Bezawada, B., & Ray, I. (2019). Typosquatting Detection Using Advanced Machine Learning Models. *Proceedings of the 18th IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 321–328.

APPENDIX – I

SOURCE CODE

Phising_site_urls.csv

```

1 URL Label
2 nobell.it/70ffb52d079109dca5664cce6f317373782/login.SkyPe.com/en/cgi-bin/verification/login/70ffb52d079109dca5664cce6f317373/index.php?cmd=_profile-ach&os
3 www.dghjdjdf.com/paypal.co.uk/cygcj-bin/websecrmds-home-customer&nav-1/loading.php,bad
4 serviciosbys.com/paypal.cgi.bin.get-info.herf.secure.dispatch35463256r-zr321654641dsf654321874/href/href/secure/center/update/limit/seccure/4d7a1ff5c5
5 mail.printakid.com/www.online.americanexpress.com/index.html,bad
6 thewhiskeydregs.com/wp-content/themes/widescreen/includes/temp/promocoessmiles/7847847824HDJNDJDSJSHD/2724782784/,bad
7 smilesvoegol.servvebs.org/voegol.php,bad
8 premierpaymentprocessing.com/includes/boleto-2via-07-2012.php,bad
9 myxxxcollection.com/v1/js/jih321/bpd.com.do/do/1.popular.php,bad
10 super1000.info/docs,bad
11 horizonsgallery.com/js/bin/ssl1/_id/www.paypal.com/fr/cgi-bin/websecr/cmds_registration-run/login.php?cmd=_login-run&dispatch=1471c4b044ae2be9e2fc3ec
12 phleblog.com.ua/libraries/joomla/results.php,bad
13 docs.google.com/spreadsheet/viewform?formkey=d5rVedSV2p8dkpS8y11V3o2eDwbnc6NQ,bad
14 www.colincole.com.br/Scripts/smiles/jpt-br/Paginas/default.aspx,bad
15 www.henkdeinumboomwekerij.nl/language/pdf_fonts/smiles.php,bad
16 perfectsolutionofall.net/wp-content/themes/twentyten/wiresource/,bad
17 lingshc.com/old_aol.1.3/?login=&lis=10&ligertID=1993745&us=1,bad
18 anonymidentity.net/remax/remax.htm,bad
19 dutchweb.gthpost.com/zimbrea/exch/owa/uleth/index.html,bad
20 www.avedeairo.com/site/plugins/chase/,bad
21 aslaidconcentration.com/papikuk1/websecrmds-home-customer&nav-1/,bad
22 www.regaranch.info/grafika/file/2012/atualizacao/www.itau.com.br/,bad
23 optimistic-pessimism.com/aoluserupdatealert.info.htm,bad
24 mercadolivre.com.br/premiosfidelidade2012.com.br/confirmar/,bad
25 www.everythinggoingon.net/-gpeveryt/home/Email/,bad
26 mercadolivre.com.br/premiosfidelidade2012.com.br/,bad
27 www.revitolcream.org/wp-content/plugins/all-in-one-seo-pack/rex/secure-code17/security/,bad
28 jameshowardmusic.com/wp-content/themes/widescreen/includes/cache/bbnew/tb.php,bad
29 xini.eu/000q,bad
30 myxxxcollection.com/v1/js/555klisdr/bpd.com.do/do/1.popular.php,bad
31 paypal.com/cgi-bin.websecr.cmd.login.submit.dispatch.5885d80a13c03fae8d8bcd55a50598f04d34b4bf5tt1.mediareso.com/secure-code90/security/,bad
32 ebayisapidld.altervista.org/,bad
33 horizonsgallery.com/js/bin/ssl1/_id/www.paypal.com/fr/cgi-bin/websecr/cmds_registration-run/login.php?cmd=_login-run&dispatch=7adb8989b4614aec6715340ae31ac
34 www.revitolcream.org/wp-content/plugins/all-in-one-seo-pack/rex/secure-code5/security/login.php,bad
35 sentemeda.altervista.org/paypinko/paypinko/procesing.php,bad
36 promusic.co/components/interbank.com/,bad
37 uvb.lg.ua/templates/a216/.PayPal/e9d83d4d35ff9d702196ab514573999/,bad
38 docs.google.com/a/umsa.edu.pe/spreadsheet/viewform?formkey=dEzo0XQwTzFkaUJCRtdhVIA3cTayV1E6Q,bad
39 remax.com.behinehsazerooyesh.com/remax/index.htm,bad
40 www.aciaydance.com/ncpf.php,bad
  
```

Model development (Jupyter notebook)

```

import pandas as pd
import numpy as np
import re

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report
from scipy.sparse import hstack
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('dataset/phishing_site_urls.csv')
df.head()

URL Label
0 nobell.it/70ffb52d079109dca5664cce6f317373782/... bad
1 www.dghjdjdf.com/paypal.co.uk/cygcj-bin/websecr... bad
2 serviciosbys.com/paypal.cgi.bin.get-info.herf... bad
3 mail.printakid.com/www.online.americanexpress... bad
4 thewhiskeydregs.com/wp-content/themes/widescr... bad

bad_df = df[df['Label'] == 'bad'].sample(30000, random_state=42)
good_df_original = df[df['Label'] == 'good'].sample(30000, random_state=42)

df = pd.concat([bad_df, good_df_original], ignore_index=True)
  
```

```
File Edit Selection View Go Run Terminal Help
phishing_main.ipynb X
# ULTIMATE TEST SET (REAL + TRICKY + EDGE CASES)
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.14.4

good_urls = [
    "google.com", "youtube.com", "facebook.com", "amazon.com", "amazon.in", "amazon.co.uk",
    "wikipedia.org", "twitter.com", "instagram.com", "linkedin.com", "netflix.com",
    "microsoft.com", "apple.com", "github.com", "stackoverflow.com", "paypal.com",
    "paypal.me", "reddit.com", "yahoo.com", "bing.com", "openai.com", "flipkart.com",
    "irctc.co.in", "hotstar.com", "primevideo.com", "whatsapp.com", "telegram.org",
    "zoom.us", "coursera.org", "udemy.com", "medium.com", "quora.com"
]

good_df = pd.DataFrame({
    "URL": good_urls,
    "Label": ["good"] * len(good_urls)
})

df = pd.concat([df, good_df], ignore_index=True)

# SHUFFLE DATA
df = df.sample(frac=1).reset_index(drop=True)

df['clean_url'] = df['URL'].str.lower().apply(
    lambda x: re.sub(r'^https?://(www\.)?', '', x)
)

from sklearn.feature_extraction.text import TfidfVectorizer

cv = TfidfVectorizer(
    ngram_range=(1,2),
    max_features=3000,
    min_df=2,
    max_df=0.9
)
```

```
File Edit Selection View Go Run Terminal Help
phishing_main.ipynb X
# ULTIMATE TEST SET (REAL + TRICKY + EDGE CASES)
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.14.4

from sklearn.feature_extraction.text import TfidfVectorizer

cv = TfidfVectorizer(
    ngram_range=(1,2),
    max_features=3000,
    min_df=2,
    max_df=0.9
)

text_features = cv.fit_transform(df['clean_url'])

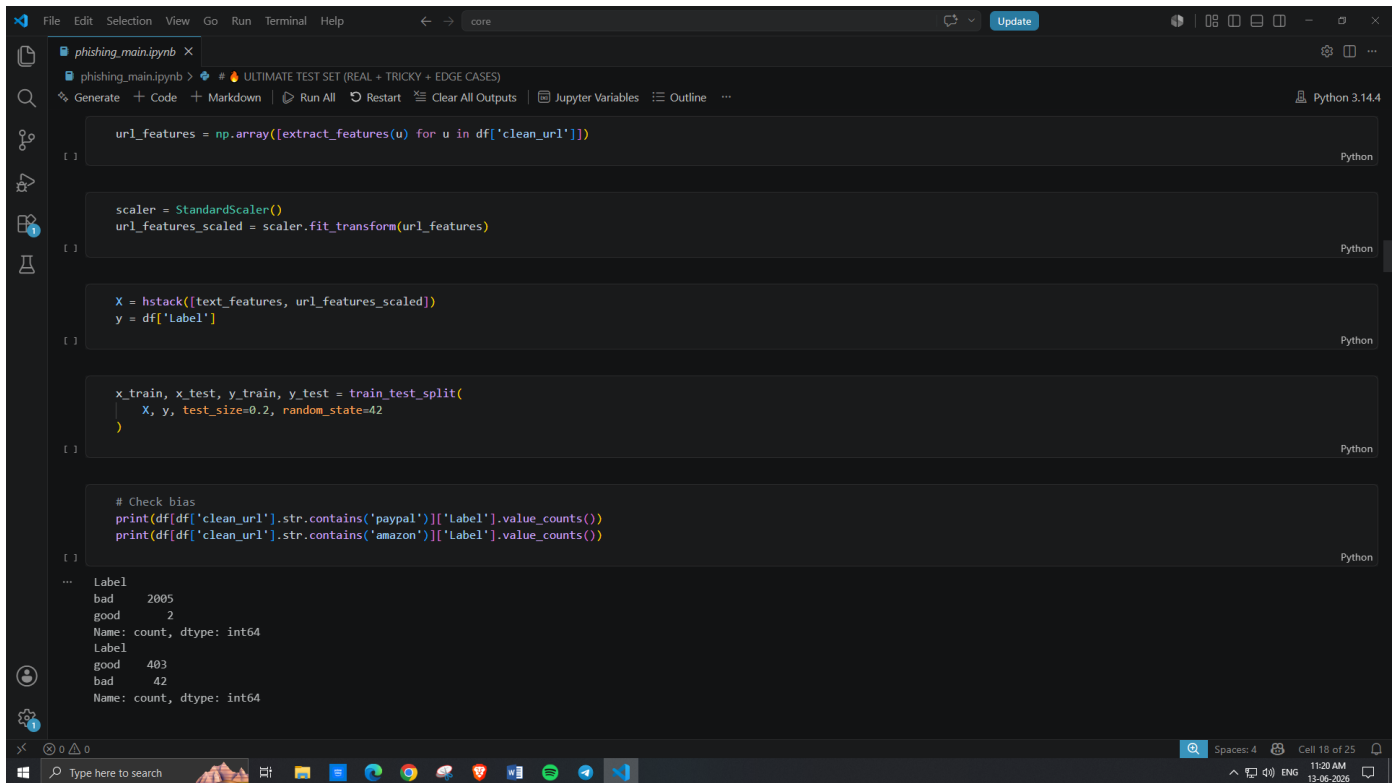
import difflib

def is_lookalike(domain):
    brands = ['google', 'amazon', 'paypal', 'facebook', 'microsoft', 'apple']

    for b in brands:
        ratio = difflib.SequenceMatcher(None, domain, b).ratio()
        if 0.75 < ratio < 1.0:
            return 1
    return 0

def extract_features(url):
    url = url.lower()
    domain = url.split('/')[0]
    parts = domain.split('.')
    base_domain = parts[0]

    # scam keywords (already good)
    scam_words = ['free', 'money', 'win', 'prize', 'gift', 'bonus', 'offer']
    scam_count = sum(word in url for word in scam_words)
```

The screenshot shows a Jupyter Notebook with the following code cells:

```
url_features = np.array([extract_features(u) for u in df['clean_url']])

scaler = StandardScaler()
url_features_scaled = scaler.fit_transform(url_features)

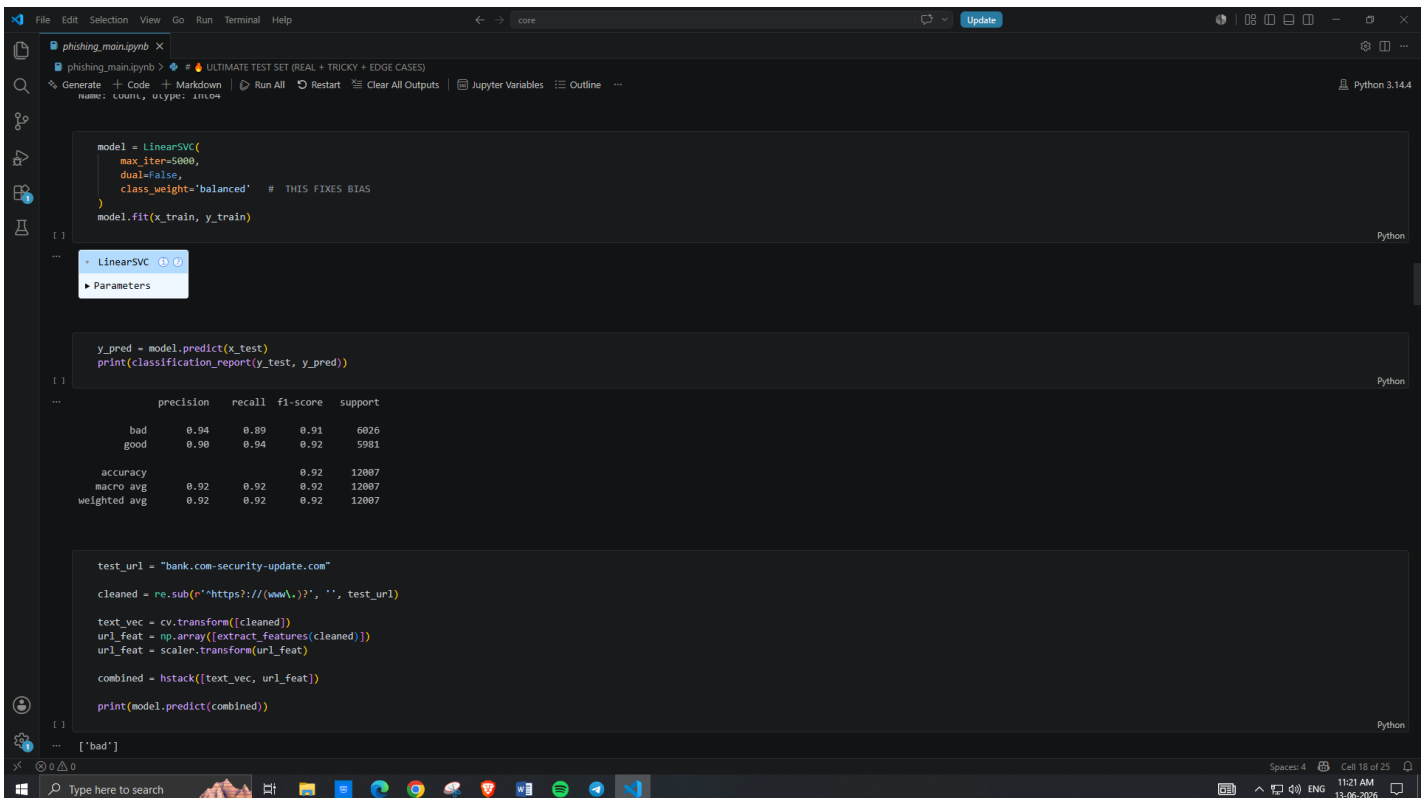
X = hstack([text_features, url_features_scaled])
y = df['Label']

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Check bias
print(df[df['clean_url'].str.contains('paypal')]['Label'].value_counts())
print(df[df['clean_url'].str.contains('amazon')]['Label'].value_counts())
```

The output of the last cell shows the distribution of labels for URLs containing 'paypal' and 'amazon':

```
Label
bad      2805
good       2
Name: count, dtype: int64
Label
good     403
bad       42
Name: count, dtype: int64
```



The screenshot shows a Jupyter Notebook with the following code cells:

```
model = LinearSVC(
    max_iter=5000,
    dual=False,
    class_weight='balanced' # THIS FIXES BIAS
)
model.fit(x_train, y_train)

y_pred = model.predict(x_test)
print(classification_report(y_test, y_pred))
```

The output of the second cell shows the classification report:

	precision	recall	f1-score	support
bad	0.94	0.89	0.91	6026
good	0.90	0.94	0.92	5981
accuracy			0.92	12007
macro avg	0.92	0.92	0.92	12007
weighted avg	0.92	0.92	0.92	12007

```
test_url = "bank.com-security-update.com"
cleaned = re.sub(r"https?://(www\.)?", '', test_url)

text_vec = cv.transform([cleaned])
url_feat = np.array([extract_features(cleaned)])
url_feat = scaler.transform(url_feat)

combined = hstack([text_vec, url_feat])
print(model.predict(combined))
```

The output of the last cell is:

```
['bad']
```

App.Py

```

File Edit Selection View Go Run Terminal Help
core
app.py
app.py:~
1 from flask import Flask, render_template, request, jsonify # added jsonify
2 from flask_cors import CORS # added CORS
3 import pickle
4 import re
5 import numpy as np
6 from scipy.sparse import hstack, csr_matrix
7 import difflib
8 import shap
9 import os
10 from dotenv import load_dotenv
11 import hashlib
12 from google import genai
13 from google.genai import types
14
15 load_dotenv()
16
17 app = Flask(__name__)
18 CORS(app) # allows Chrome extension to call your Flask server
19
20 # ===== LOAD FILES =====
21 with open("vectorizernew.pkl", "rb") as f:
22     vectorizer = pickle.load(f)
23
24 with open("phishing_modelnew.pkl", "rb") as f:
25     model = pickle.load(f)
26
27 with open("scalernew.pkl", "rb") as f:
28     scaler = pickle.load(f)
29
30 # ===== GEMINI SETUP =====
31 GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY", "")
32 if GEMINI_API_KEY:
33     gemini_client = genai.Client(api_key=GEMINI_API_KEY)
34     print("✓ Gemini API connected successfully.")
35 else:
36     gemini_client = None
37     print("⚠ GEMINI_API_KEY not set - Gemini explanations will be disabled.")
38
39 # ===== GEMINI RESPONSE CACHE =====
40 _gemini_cache = dict()
41
42 # ===== SAFE DOMAINS =====
43 safe_domains = [
44     "google.com", "amazon.com", "amazon.in", "amazon.co.uk",
45     "paypal.com", "paypal.co.uk",
46     "github.com", "facebook.com", "linkedin.com"
47 ]

```

```

File Edit Selection View Go Run Terminal Help
core
app.py
app.py:~
78
79 # ===== LOOKALIKE =====
80 def is_lookalike(domain):
81     brands = ['google', 'amazon', 'paypal', 'facebook', 'microsoft', 'apple']
82     for b in brands:
83         ratio = difflib.SequenceMatcher(None, domain, b).ratio()
84         if 0.70 < ratio < 1.0:
85             return 1
86     return 0
87
88 # ===== FREE HOST CHECK =====
89 def is_free_host(domain):
90     parts = domain.split('.')
91     if len(parts) >= 2:
92         root = '.'.join(parts[-2:])
93         if root in SUSPICIOUS_FREE_HOSTS:
94             return True
95     return False
96
97 # ===== FEATURE EXTRACTION =====
98 def extract_features(url):
99     url = url.lower()
100     domain = url.split('/')[0]
101     parts = domain.split('.')
102     base_domain = parts[0]
103
104     scam_words = ['free', 'money', 'win', 'prize', 'gift', 'bonus', 'offer', 'giveaway']
105     scam_count = sum(word in url for word in scam_words)
106
107     phrase_patterns = [
108         'account-verification', 'verification-required', 'user-verification',
109         'limited-offer', 'offer-free', 'free-subscription',
110         'confirm-account', 'update-details', 'login-support', 'security-alert'
111     ]
112     phrase_count = sum(p in url for p in phrase_patterns)
113
114     return [
115         len(url), url.count('.'), url.count('-'),
116         sum(c.isdigit() for c in url),
117         int('login' in url), int('secure' in url),
118         int('verify' in url), int('account' in url),
119         int('update' in url), int(url.count('.') > 4),
120         int('@' in url), int(url.count('-') > 2),
121         int(len(parts) > 4),
122         int(domain.endswith('.xyz', '.tk', '.ml', '.ga'))),
123         is_lookalike(base_domain),

```

```

File Edit Selection View Go Run Terminal Help
core
app.py
app.py > ...
138
139
140 # ===== SHAP EXPLANATION =====
141 def get_shap_explanation(combined_input, url_features_raw, prediction):
142     if hasattr(combined_input, "toarray"):
143         combined_dense = combined_input.toarray()
144     else:
145         combined_dense = np.array(combined_input)
146
147     shap_values = explainer.shap_values(combined_dense)
148     handcrafted_shap = shap_values[0, -n_handcrafted:]
149     show_direction = "phishing" if prediction == "bad" else "safe"
150
151     explanations = []
152     for name, shap_val, raw_val in zip(feature_names, handcrafted_shap, url_features_raw[0]):
153         if abs(shap_val) <= 0.01:
154             continue
155         is_continuous = name in CONTINUOUS_FEATURES
156         if shap_val > 0:
157             if show_direction != "phishing":
158                 continue
159             if is_continuous and raw_val > 0:
160                 explanations.append(make_item(name, shap_val, raw_val, "phishing"))
161             elif not is_continuous and raw_val == 1:
162                 explanations.append(make_item(name, shap_val, raw_val, "phishing"))
163         else:
164             if show_direction != "safe":
165                 continue
166             if is_continuous:
167                 explanations.append(make_item(name, shap_val, raw_val, "safe"))
168             elif raw_val == 0:
169                 explanations.append(make_item(name, shap_val, raw_val, "safe"))
170
171     explanations.sort(key=lambda x: abs(x["shap"]), reverse=True)
172     return explanations[:5]
173
174
175 def make_item(name, shap_val, raw_val, direction):
176     return {
177         "name": name,
178         "shap": shap_val,
179         "raw": raw_val,
180         "direction": direction
181     }

```

```

File Edit Selection View Go Run Terminal Help
core
app.py
app.py > ...
184 # ===== GEMINI EXPLANATION =====
185 def get_gemini_explanation(url, prediction, shap_signals, free_host=False):
186     if not gemini_client:
187         return None
188
189     verdict_text = "PHISHING" if prediction == "bad" else "SAFE"
190     cache_key = hashlib.md5(f"{url.lower()}:{verdict_text}".encode()).hexdigest()
191
192     if cache_key in _gemini_cache:
193         return _gemini_cache[cache_key]
194
195     try:
196         top_signals = shap_signals[:2]
197         if top_signals:
198             signals_text = ", ".join(
199                 f"[{s['name']}] ({s['Risk'] if s['direction']=='phishing' else 'Safe'})"
200                 for s in top_signals
201             )
202         else:
203             signals_text = "free hosting" if free_host else "URL patterns"
204
205         prompt = (
206             f"URL: {url} | Result: {verdict_text} | Flags: {signals_text}\n"
207             f"Explain in 2 sentences why it is {verdict_text}."
208         )
209
210         response = gemini_client.models.generate_content(
211             model="gemini-2.5-flash-lite",
212             contents=prompt,
213             config=types.GenerateContentConfig(
214                 temperature=0.3,
215                 max_output_tokens=80
216             )
217         )
218
219         result = response.text.strip()
220         _gemini_cache[cache_key] = result
221         return result
222     except Exception:
223         return None

```

```

File Edit Selection View Go Run Terminal Help
app.py X
app.py > ...
227 # ===== PREDICT =====
228 def smart_predict(url):
229     cleaned = re.sub(r"(https?://www.)?", "", url.lower())
230     domain = cleaned.split('/')[0]
231
232     for safe in safe_domains:
233         if domain == safe or domain.endswith('.' + safe):
234             gemini_text = get_gemini_explanation(url, "good", [])
235             return "good", [], gemini_text
236
237     if is_free_host(domain):
238         gemini_text = get_gemini_explanation(url, "bad", [], True)
239         return "bad", [], gemini_text
240
241     text_vec = vectorizer.transform([cleaned])
242     url_feat_raw = np.array([extract_features(cleaned)])
243     url_feat_scaled = scaler.transform(url_feat_raw)
244     combined = hstack([text_vec, url_feat_scaled])
245
246     prediction = model.predict(combined)[0]
247     shap_signals = get_shap_explanation(combined, url_feat_raw, prediction)
248     gemini_text = get_gemini_explanation(url, prediction, shap_signals)
249
250     return prediction, shap_signals, gemini_text
251
252
253 # ===== MAIN ROUTE (web app) =====
254 @app.route("/", methods=['GET', 'POST'])
255 def index():
256     result = None
257     explanation = []
258     gemini_text = None
259     url_input = ""
260
261     if request.method == "POST":
262         url = request.form["url"].strip()
263         url_input = url
264
265         prediction, explanation, gemini_text = smart_predict(url)
266
267         if prediction == "bad":
268             result = "⚠️ Phishing Website Detected"
269         else:
270             result = "✅ Safe Website"
271
272     return render_template(
273         "index.html",
274         result=result,
275         url_input=url_input,
276         explanation=explanation,
277         gemini_text=gemini_text
278     )

```

```

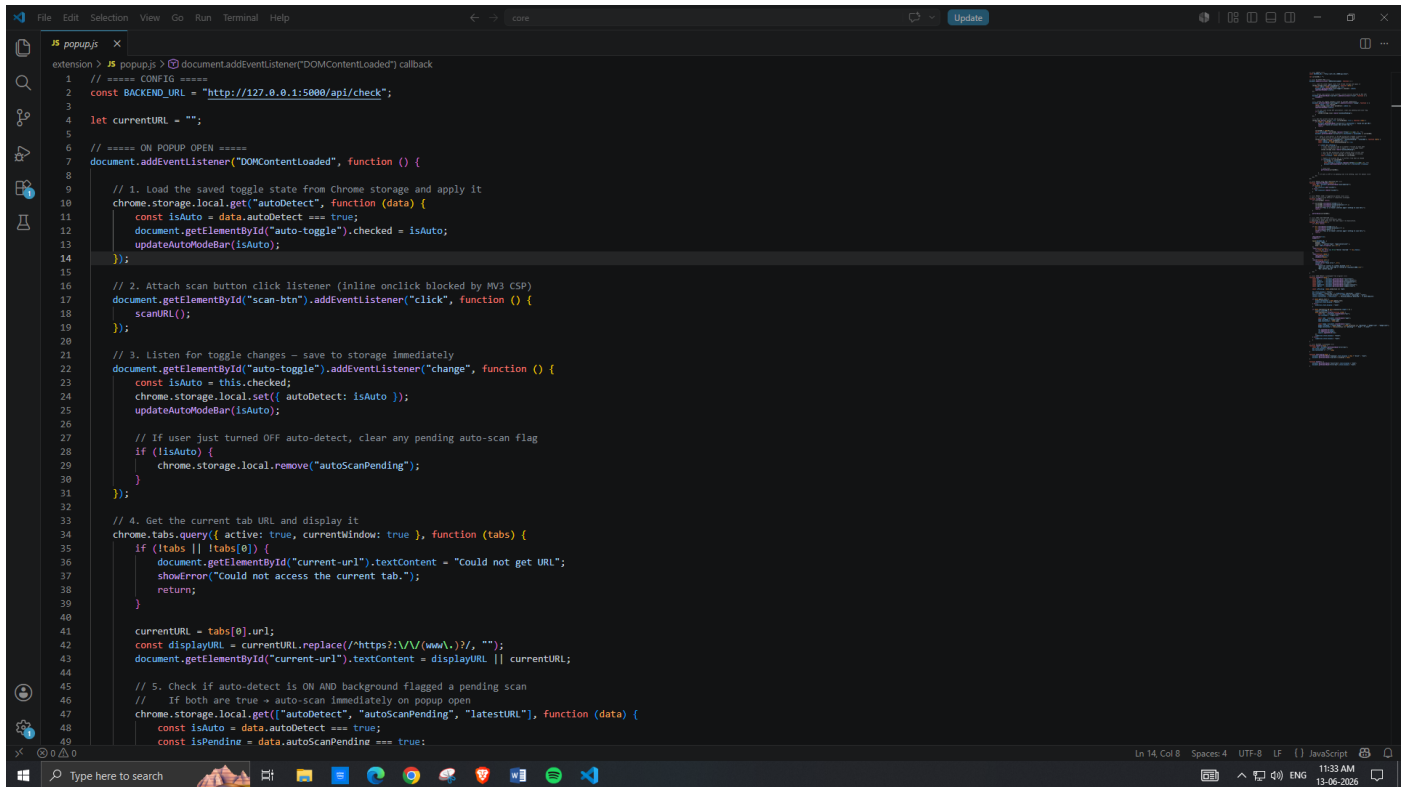
File Edit Selection View Go Run Terminal Help
app.py X
app.py > ...
281 # ===== API ROUTE (Chrome extension) =====
282 @app.route("/api/check", methods=['POST'])
283 def api_check():
284     """
285     JSON endpoint for the Chrome extension.
286     Receives: { "url": "https://example.com" }
287     Returns: { "prediction": "good"/"bad", "explanation": [...], "gemini_text": "..." }
288     """
289     data = request.get_json()
290     if not data or "url" not in data:
291         return jsonify({"error": "No URL provided"}), 400
292
293     url = data["url"].strip()
294
295     try:
296         prediction, explanation, gemini_text = smart_predict(url)
297         return jsonify({
298             "prediction": prediction,
299             "explanation": explanation,
300             "gemini_text": gemini_text
301         })
302     except Exception as e:
303         return jsonify({"error": str(e)}), 500
304
305 if __name__ == "__main__":
306     app.run(debug=True)
307
308

```

Browser extension

```
File Edit Selection View Go Run Terminal Help
background.js
extension > JS background.js > ...
14
15 const BACKEND_URL = "http://127.0.0.1:5000/api/check";
16
17 // ===== LISTEN FOR TAB UPDATES =====
18 // status === "complete" means the page fully loaded - not just started loading.
19 // We ignore "loading" status to avoid scanning incomplete pages.
20 chrome.tabs.onUpdated.addListener(function(tabId, changeInfo, tab) {
21   if (changeInfo.status !== "complete") return;
22   if (!tab.url) return;
23
24   // Skip Chrome internal pages - can't scan these
25   if (tab.url.startsWith("chrome://") ||
26       tab.url.startsWith("chrome-extension://") ||
27       tab.url.startsWith("about:") ||
28       tab.url.startsWith("edge://")) {
29     return;
30   }
31
32   // Check if auto-detect is enabled before doing anything
33   chrome.storage.local.get("autoDetect", function(data) {
34     const autoDetectOn = data.autoDetect === true;
35     if (!autoDetectOn) return;
36
37     // Store the latest URL and clear any old result
38     // The popup will read this when it opens and auto-scan
39     chrome.storage.local.set({
40       latestURL: tab.url,
41       latestTabId: tabId,
42       autoScanPending: true // flag tells popup to auto-scan on open
43     });
44   });
45 });
46
47 // ===== LISTEN FOR TAB ACTIVATION =====
48 // When user switches to a different tab, update the stored URL
49 // so the popup shows the correct page when opened
50 chrome.tabs.onActivated.addListener(function(activeInfo) {
51   chrome.tabs.get(activeInfo.tabId, function(tab) {
52     if (!tab || !tab.url) return;
53     if (tab.url.startsWith("chrome://") ||
54         tab.url.startsWith("chrome-extension://") ||
55         tab.url.startsWith("about:")) {
56       return;
57     }
58
59     chrome.storage.local.get("autoDetect", function(data) {
60       if (data.autoDetect !== true) return;
61       chrome.storage.local.set({
62         latestURL: tab.url,
```

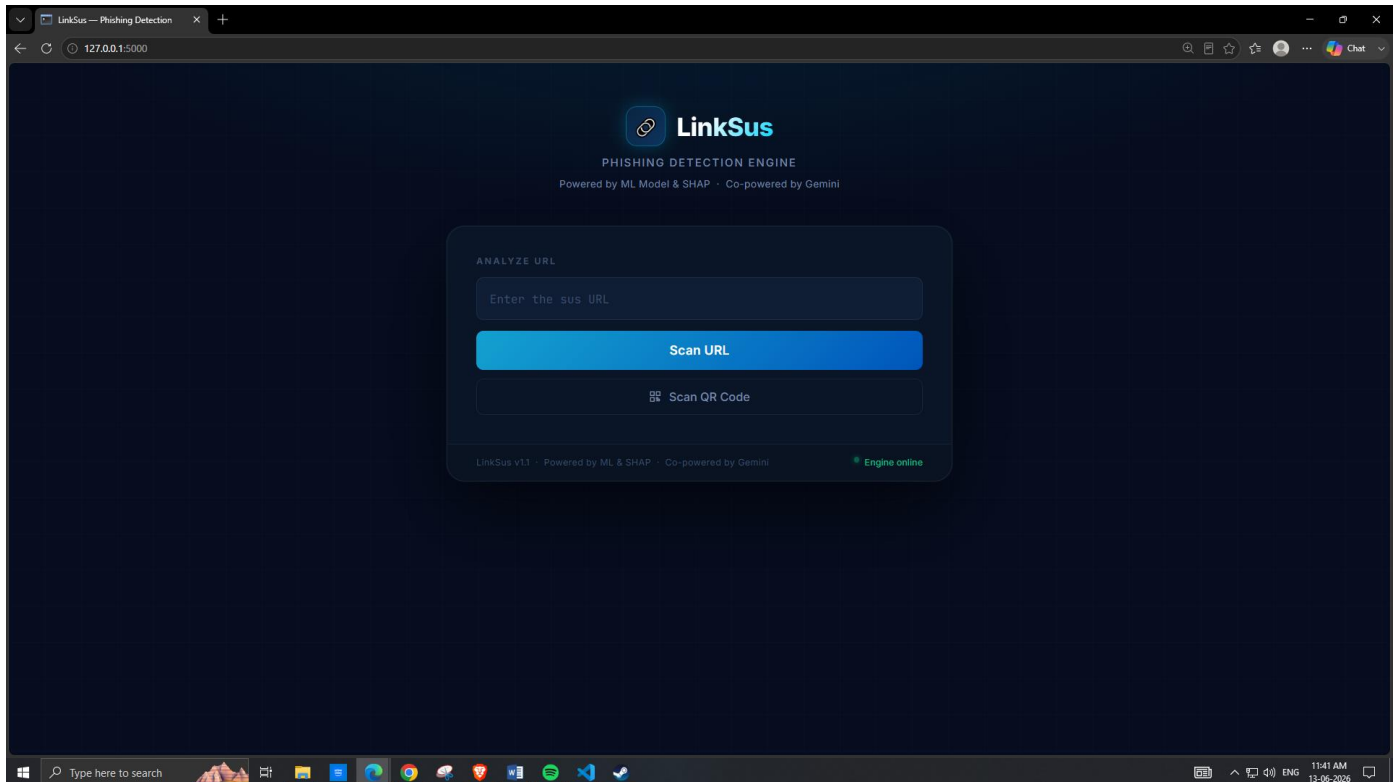
```
File Edit Selection View Go Run Terminal Help
manifest.json
extension > {} manifest.json > ...
1
2 {
3   "manifest_version": 3,
4   "name": "LinkSus",
5   "version": "1.1",
6   "description": "Smart Phishing Detection - checks if the current website is safe or phishing using AI.",
7
8   "permissions": [
9     "activeTab",
10    "tabs",
11    "storage"
12  ],
13
14   "background": {
15     "service_worker": "background.js"
16   },
17
18   "action": {
19     "default_popup": "popup.html",
20     "default_title": "LinkSus - Phishing Detector"
21   },
22
23   "icons": {
24     "16": "icon16.png",
25     "48": "icon48.png",
26     "128": "icon128.png"
27   }
28 }
```



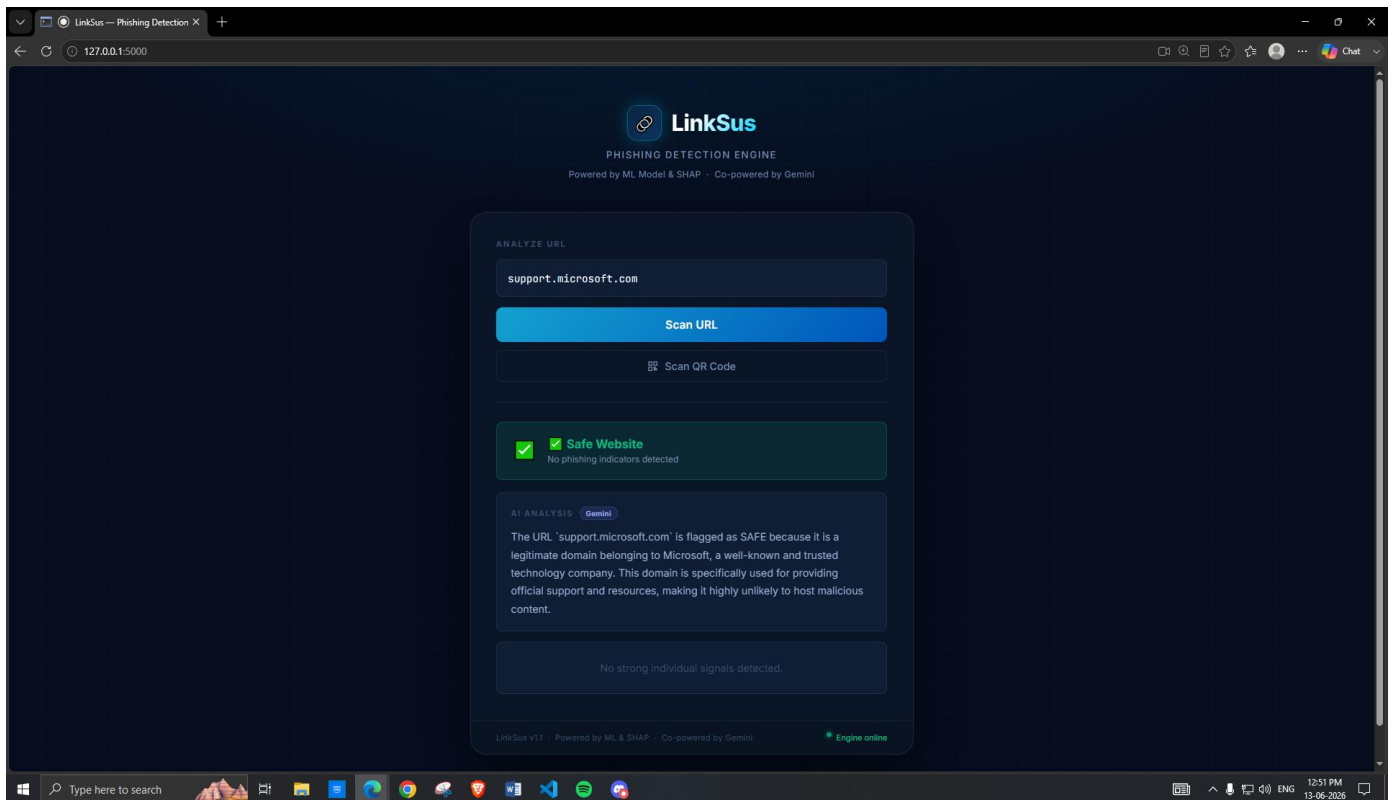
```
extension > JS popups.js > document.addEventListener("DOMContentLoaded") callback
1 // ===== CONFIG =====
2 const BACKEND_URL = "http://127.0.0.1:5000/api/check";
3
4 let currentURL = "";
5
6 // ===== ON POPUP OPEN =====
7 document.addEventListener("DOMContentLoaded", function () {
8
9     // 1. Load the saved toggle state from Chrome storage and apply it
10    chrome.storage.local.get("autoDetect", function (data) {
11        const isAuto = data.autoDetect === true;
12        document.getElementById("auto-toggle").checked = isAuto;
13        updateAutoModeBar(isAuto);
14    });
15
16    // 2. Attach scan button click listener (inline onclick blocked by MV3 CSP)
17    document.getElementById("scan-btn").addEventListener("click", function () {
18        scanURL();
19    });
20
21    // 3. Listen for toggle changes – save to storage immediately
22    document.getElementById("auto-toggle").addEventListener("change", function () {
23        const isAuto = this.checked;
24        chrome.storage.local.set({ autoDetect: isAuto });
25        updateAutoModeBar(isAuto);
26
27        // If user just turned OFF auto-detect, clear any pending auto-scan flag
28        if (!isAuto) {
29            chrome.storage.local.remove("autoScanPending");
30        }
31    });
32
33    // 4. Get the current tab URL and display it
34    chrome.tabs.query({ active: true, currentWindow: true }, function (tabs) {
35        if (!tabs || !tabs[0]) {
36            document.getElementById("current-url").textContent = "Could not get URL";
37            showFnrn("Could not access the current tab.");
38            return;
39        }
40
41        currentURL = tabs[0].url;
42        const displayURL = currentURL.replace(/https?:\/\/(www\.)?/, "");
43        document.getElementById("current-url").textContent = displayURL || currentURL;
44
45        // 5. Check if auto-detect is ON AND background flagged a pending scan
46        // If both are true → auto-scan immediately on popup open
47        chrome.storage.local.get(["autoDetect", "autoScanPending", "latestURL"], function (data) {
48            const isAuto = data.autoDetect === true;
49            const isPending = data.autoScanPending === true;
```

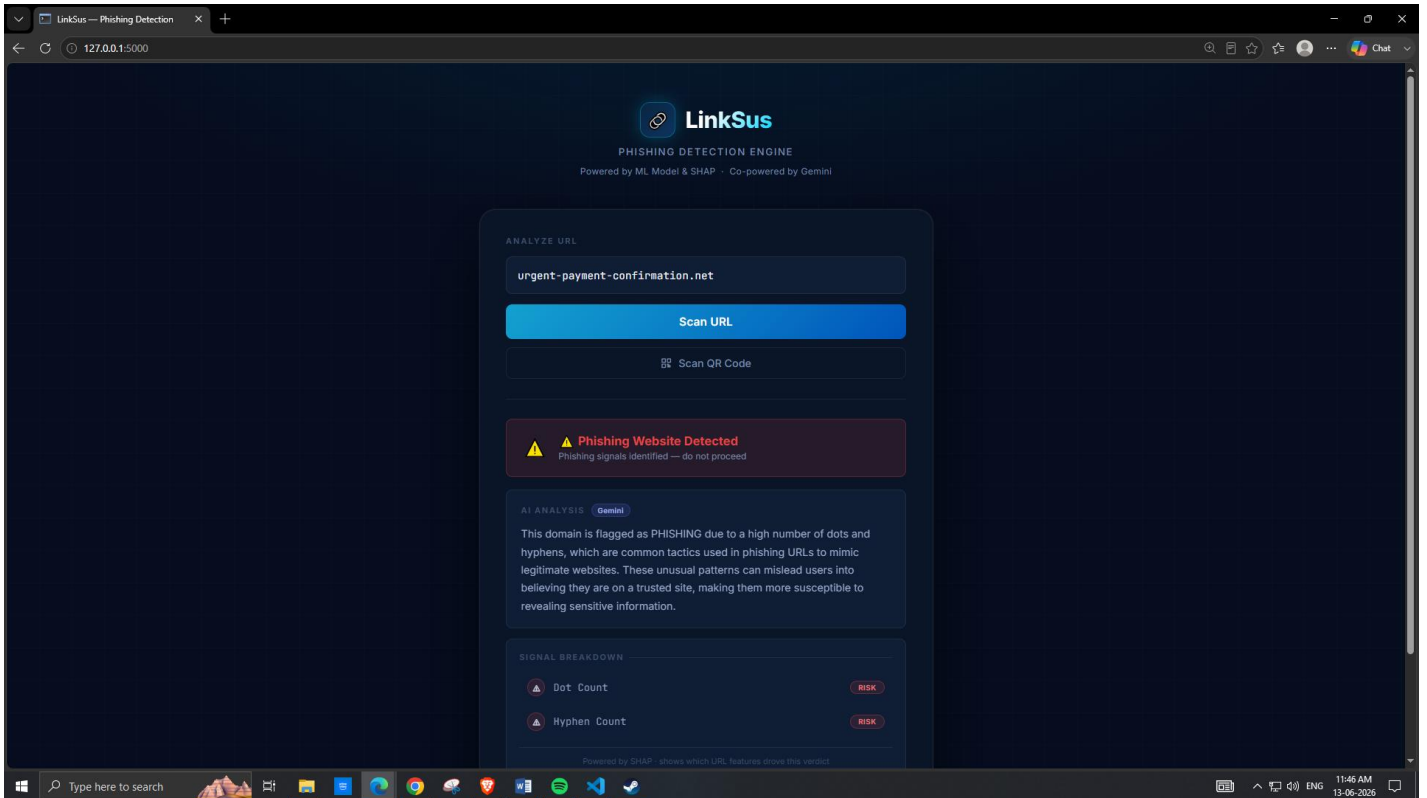
APPENDIX – II FINAL RESULTS

Main App

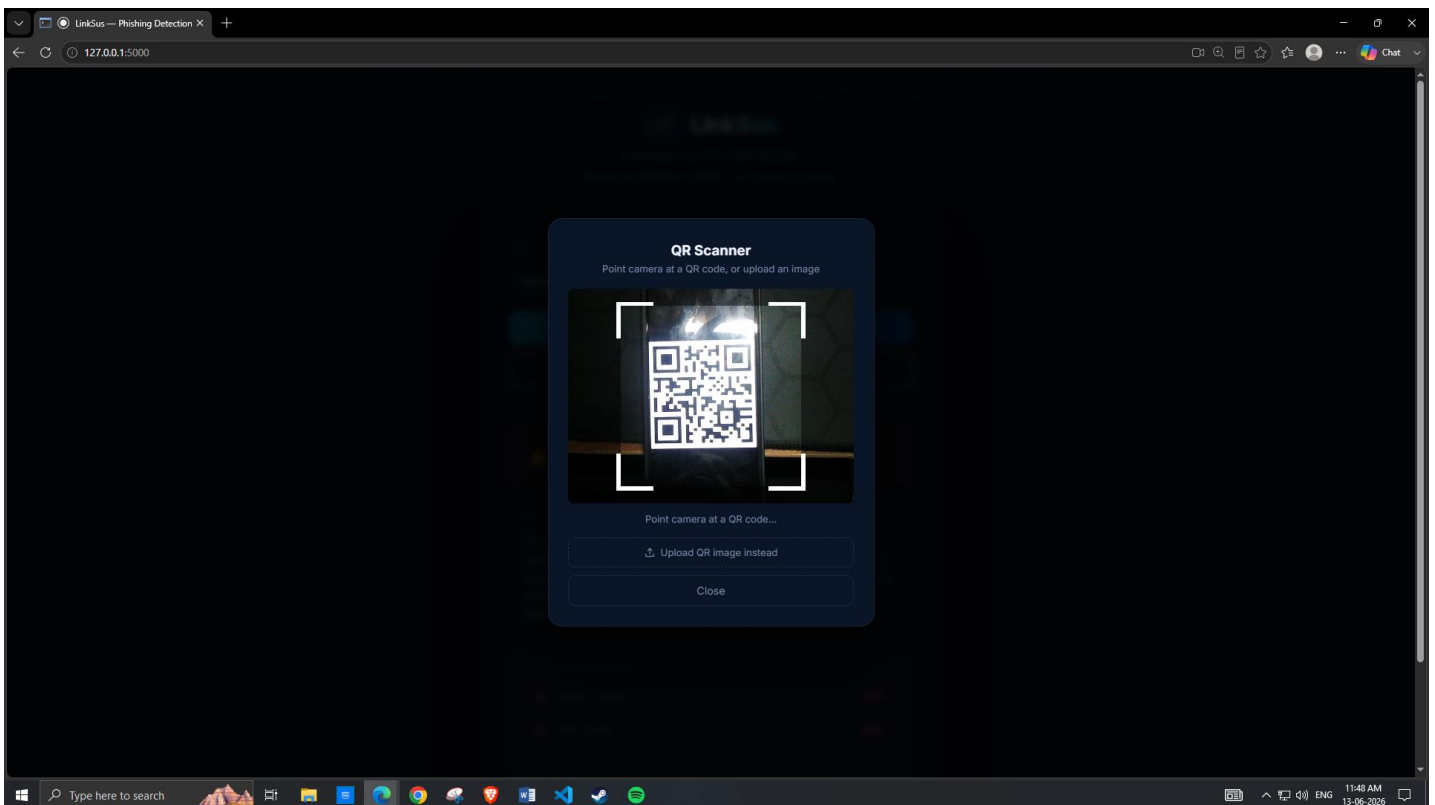


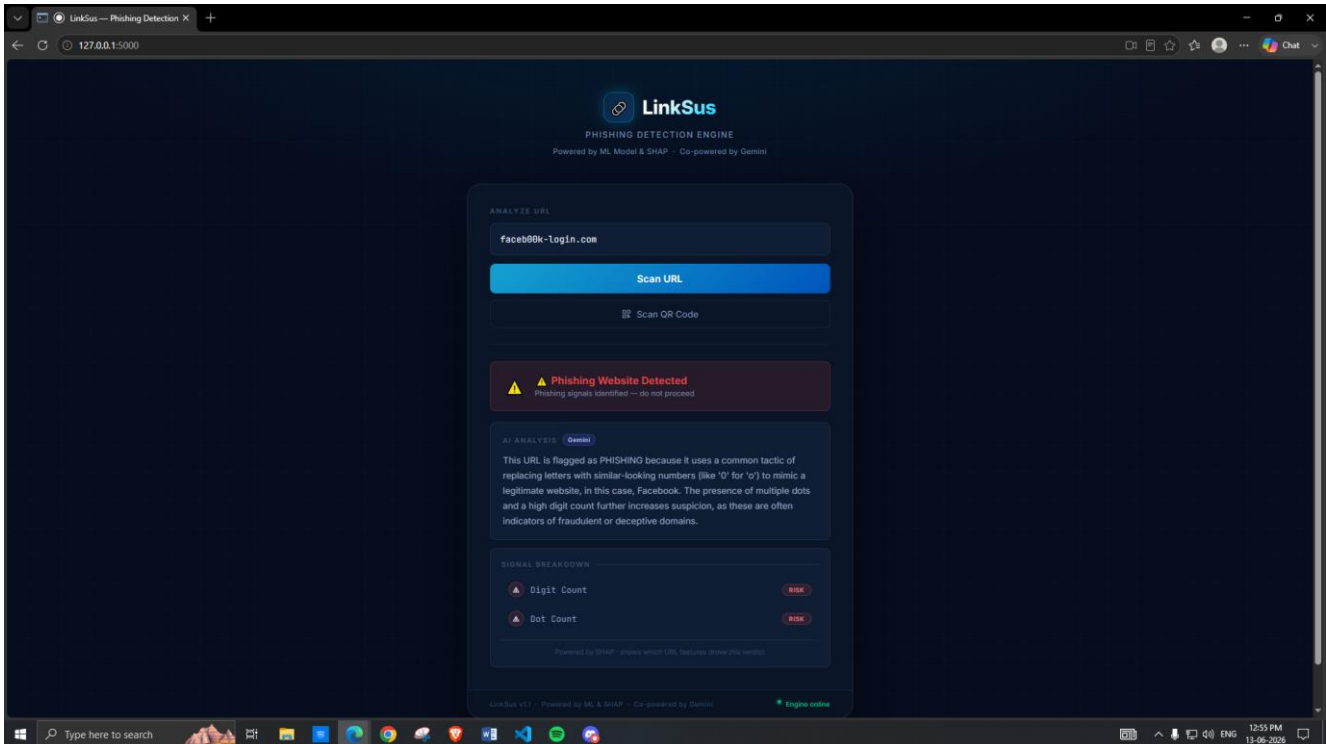
Output





QR scanner





Browser extension

